C | 数据与日志

一个程序能接收请求、运行函数并返回 JSON，事情还没有结束：程序重启以后，原来的预约还在吗？两张表分别记着客户与预约，怎样知道它们对应的是同一个人？确认预约需要改三处数据，如果第三步失败，前两步算不算成功？

本册解决的是这些问题。数据库不是把 Python 字典换一个存放位置，而是让长期事实有共同结构，让读取有明确含义，让修改有完整边界。日志则留下“哪次操作得到了什么结果”的线索，帮助区分数据错误、访问失败和读取太慢。

先用几条看得见的记录学习查询，再从查询需要反推表的设计；随后处理写入、应用访问和性能。例子是一套社区维修预约业务：客户提出工单，预约占用技术员的一个时段。每章给出本章所需的事实，不要求先拼成一套服务才能阅读。

第一次只看这里

直接进入 C01。第一次只追一张表：哪些事实被保存、查询怎样选出结果、日志怎样帮助复查。

本册的详细关系图、章节路线和查用说明放在全书后部。遇到‘这一步和前后哪里相连’时再回看，不需要在第一轮先读完。

目录

第一次只看这里 1

C01 | 查询为什么得到这些行，而不是另外几行？ 3

C02 | 多张表怎样组成一个答案，又为什么多了或少了行？ 12

C03 | 怎样让坏数据进不来 21

C04 | 几步修改怎样一起成功或撤回 31

C05 | 应用怎样安全读到预约，并留下能追踪的线索 43

C06 | 结果对了以后，怎样判断慢在哪里 60

知识关系总览 | 把答案、事实和运行证据放回同一张图 69

用一份完整实验把这些关系接起来 72

下一册接走的是什么？ 74

来源与查用入口 74

本册总览（需要时再回看） 75

本册知识结构总览 76

C01 | 查询为什么得到这些行，而不是另外几行？

本章先判断：你要回答的业务问题需要哪些事实，当前查询结果是否真的支持这个回答。先从一张看得见的小表和结果开始，再决定是筛选、排序、汇总，还是回到数据定义检查“看起来正确”的结果。

先决定留下哪些行、按什么顺序、交给看板哪些列；SQL 写法只是把这三个决定写出来。

维修看板只想显示最早的三个未完成预约。表里有六条记录，把它们读出来不难；真正需要判断的是：该留下哪三条，按什么顺序，交给看板哪些信息？

本章先用六条已知记录回答这个问题，再处理未知值与统计。这样遇到一条能运行的 SQL 时，就能继续判断它是否真的回答了原来的问题。

先看清楚，表里到底有什么

一张表保存同一类记录。一行是一条记录，一列是在每条记录上含义一致的字段。这里的 `appointments` 保存预约；`id` 区分预约，`starts_at` 表示预约开始时间。

先看决定筛选与顺序的四列。所有姓名都是示例人物，所有日期都是固定练习日期，不指今天的实际安排。

id	customer_name	status	starts_at
101	林青	open	2026-08-22 09:00
102	周宁	open	2026-08-22 13:30
103	陈晓	pending	2026-08-23 08:30
104	吴凡	open	2026-08-21 16:00
105	赵敏	completed	2026-08-24 10:00
106	何川	open	2026-08-22 11:00

同一批行还保存了三列补充信息。下面不是第二张数据库表，只是为了读得清楚，把宽表的其他列另列出来；通过 `id` 可以找到同一行。

id	service_type	duration_minutes	note
101	电气	30	NULL
102	水路	60	带配件
103	电气	NULL	NULL

id	service_type	duration_minutes	note
104	门锁	45	已电话确认
105	电气	30	完成
106	水路	NULL	需确认

NULL 在这里表示尚无确定值。例如 103 的时长未知，不能先算作零分钟；101 的备注未知，也不是一段内容为 NULL 的文字。稍后会看到，这个区别会改变筛选与统计结果。

“最早三个未完成预约”要做哪几个决定？

下面用 SQL 把读取要求写给 SQLite：SQL 是表达查询、建表和修改要求的语言，SQLite 是实际管理表并执行这些要求的数据库引擎。本章后面的 Python 实验通过标准库 sqlite3 把语句交给 SQLite，再拿回结果；先在这六条记录上看清语句要取得什么。

当前问题的完整口径是：从 2026-08-22 00:00 开始，状态不是 completed，按开始时间从早到晚，最多三条；看板只需要编号、开始时间和客户名。

```SQL
SELECT id, starts_at, customer_name
FROM appointments
WHERE starts_at >= '2026-08-22 00:00'
  AND status != 'completed'
ORDER BY starts_at ASC, id ASC
LIMIT 3;
```

FROM 指定输入表，WHERE 指定保留条件，SELECT 列出结果需要的列。AND 要求两边同时成立；>= 包含等于边界的情况，!= 表示不相等。这里每行的 status 都有值，“未完成”按当前业务口径就是排除 completed，不是 SQL 内置的状态概念。

ORDER BY 建立顺序：ASC 为升序，时间相同时再按 id 升序。LIMIT 3 把结果限制为最多三行。关键字大写用于读得清楚，文字值使用单引号；语句末尾的分号表示结束。这些不是 Python 的列表切片或赋值语法，而是一条 SQL 查询。

可以手工核对每个决定产生了什么变化：

```
六条预约
→ 日期与状态均符合：101、102、103、106
→ 按时间及编号排序：101、106、102、103
→ 取前三条：101、106、102
→ 显示编号、开始时间、客户名
```

于是得到：

id	starts_at	customer_name
101	2026-08-22 09:00	林青
106	2026-08-22 11:00	何川
102	2026-08-22 13:30	周宁

104 被排除是因为时间太早，105 是因为已经完成。103 满足筛选条件，只是排在第四，没有进入本次看板。**不在查询结果中，不等于原记录不存在。**

> **第一次学到这里先停。** 先能从业务问题反推出三件事：保留哪些行、按什么顺序、交给看板哪些列。`WHERE`、`ORDER BY` 和 `LIMIT` 只是把这三个决定写出来；`NULL`、JOIN、事务和索引等后续内容，等你需要解释更复杂的结果时再接上。

这条箭头路线解释结果含义，不是在断言数据库内部必须照此逐步搬动数据。引擎可以采用不同执行办法；C06 会查看它选择的实际查询计划。

`SELECT` 也没有修改原表。结果只显示三列、三行，其他列和记录仍然保留。读取答案与改变事实，是两种不同动作。

去掉排序，为什么不再能证明“最早”？

下面仍然能执行，也可能返回三条看起来合理的预约：

```sql
SELECT id, starts_at, customer_name
FROM appointments
WHERE starts_at >= '2026-08-22 00:00'
  AND status != 'completed'
LIMIT 3;
```

但它只说“限制三行”，没有要求“按时间最早”。某次运行恰好按插入顺序显示，不是查询对顺序的承诺。以后加索引、改数据或换执行计划，看到的顺序可能变化。

如果只把 `ORDER BY starts_at` 留下来，同一时刻有多条预约时，它们彼此也没有确定先后。示例再加 `id`，是因为编号能区分这几条记录。需要“最近”则可用 `DESC` 降序；方向由问题决定，不能见到排序就一律抄 `ASC`。

到这里可以检验一下：101、106、102、103 都通过筛选，为什么最终没有 103？如果回答“它不符合 WHERE”，断点发生在把筛选和限量混在一起。它符合条件；时间排序之后是第四条，LIMIT 才使它没有进入这次结果。需要修复的是这两个决定的区别，不是重背整段语法。

“没有备注”为什么不能写成普通相等比较？

想找备注尚未填写的预约，直觉上可能写：

```sql
SELECT id
FROM appointments
WHERE note = NULL
ORDER BY id;
```

结果是零行，包括 101、103 都没有被选中。原因不是数据库忘了保存它们，而是 NULL 不是一个可供普通相等比较的确定值。note = NULL 的判断结果是未知，不是真；WHERE 只保留条件为真的行。

专门判断未知值应写：

```sql
SELECT id
FROM appointments
WHERE note IS NULL
ORDER BY id;
```

这次是 101、103。反向的 IS NOT NULL 得到 102、104、105、106。

同样，duration_minutes != 30 不会把未知时长也当成“不是 30”。当前表只留下已知的 60 和 45，即 102、104；如果业务还要求列出待确认的时长，就要把这个要求写出来：

```sql
SELECT id
FROM appointments
WHERE duration_minutes != 30
   OR duration_minutes IS NULL
ORDER BY id;
```

结果是 102、103、104、106。OR 表示两侧至少一侧为真即可保留。

这不是需要机械背下整张真假表的时刻。先抓住实际影响：**未知不能冒充已知的零、空文字或“不等于某值”；是否包含未知行，要由问题明确决定。**

查一天、找一种文字，条件又怎样选？

如果想列出 8 月 22 日全天的预约，最直接的是明确时间范围：

```sql
SELECT id
FROM appointments
WHERE starts_at >= '2026-08-22 00:00'
  AND starts_at < '2026-08-23 00:00'
ORDER BY starts_at, id;
```

得到 101、106、102。上界不包含次日零点，便不会把下一天混进来。这里的日期文字全都采用等宽 `YYYY-MM-DD HH:MM`、同一时区语义、有效日期，因此文字顺序与时间顺序一致。随意混用 `8/2/26`、不同偏移量或无效日期，不能照搬这个结论；输入日期要先统一合同。

对于明确包含两个端点的数值区间，`BETWEEN` 可以更直接：

```sql
SELECT id
FROM appointments
WHERE duration_minutes BETWEEN 30 AND 45
ORDER BY id;
```

30 和 45 都包含，得到 101、104、105。未知时长不自动落在这个范围内。

若问题是“备注含有确认二字”，可以用简单文字模式：

```sql
SELECT id
FROM appointments
WHERE note LIKE '%确认%'
ORDER BY id;
```

结果为 104、106。`LIKE` 里的 `%` 匹配零个或多个字符，`_` 匹配一个字符；例如 `'_青'` 能匹配两个字的“林青”。它们是模式中的特殊记号，不是 Python 乘法，也不是完整正则表达式。只要精确匹配某个状态，使用 `=` 就更清楚；没有必要把所有条件写成模糊搜索。

当 AND 与 OR 同时出现，括号应把业务分组表达清楚。比如“状态是 open 或 pending，而且从 8 月 22 日开始”，可以写：

```sql
SELECT id
FROM appointments
WHERE (status = 'open' OR status = 'pending')
  AND starts_at >= '2026-08-22 00:00'
ORDER BY starts_at, id;
```

结果为 101、106、102、103。没有括号时，AND 比 OR 先结合，可能使时间限制只约束 pending 那一侧，连较早的 open 预约 104 也放进来。选择条件先说清楚“哪些要求一起成立”，再写符号。

从“列出预约”变成“估算总工时”，结果的一行变了什么？

只取编号 101、102、103，它们的时长是 30、60、NULL。现在需要问的不是“哪一条”，而是这组记录的数量和已知工时。

```sql
SELECT COUNT(*) AS appointment_count,
       COUNT(duration_minutes) AS known_count,
       SUM(duration_minutes) AS total_minutes,
       AVG(duration_minutes) AS average_minutes,
       MIN(duration_minutes) AS shortest,
       MAX(duration_minutes) AS longest
FROM appointments
WHERE id BETWEEN 101 AND 103;
```

AS 给结果列一个清晰名字。这里各列的答案是：

结果列	值	回答的具体问题
appointment_count	3	有几条预约，包括时长未知的预约
known_count	2	几条预约有已知时长
total_minutes	90	已知时长的和是多少
average_minutes	45.0	已知时长的平均值是多少，即 90 ÷ 2
shortest	30	最短的已知时长是多少

结果列	值	回答的具体问题
longest	60	最长的已知时长是多少

`COUNT(*)` 数行，`COUNT(列)` 数非 `NULL` 的值。这里的 `SUM`、`AVG`、`MIN`、`MAX` 也忽略 `NULL`，因此 90 不是三条预约的完整总工时，只是已知部分。把平均数说成 30，相当于擅自把未知时长算作零，改了数据含义。

聚合后的这一行代表整组记录的统计，不代表某一条预约。 因此不能随手再塞一个 `customer_name` 进去，期待它代表整组所有客户。下一章需要“每位技术员一行”时，会先划分组再统计。

如果筛选后根本没有行，这种不带分组的查询仍给一行统计：两个 `COUNT` 为 0，`SUM`、`AVG`、`MIN`、`MAX` 为 `NULL`。这与“有记录且已知总量确实为零”不同。

只是想知道有哪些服务类型，需要统计吗？

```sql
SELECT DISTINCT service_type
FROM appointments
ORDER BY service_type;
```

结果集合只有电气、水路、门锁三种，具体显示次序由排序规则决定。`DISTINCT` 使结果里完全相同的行只留一份。

若改为 `SELECT DISTINCT service_type, status`，去重对象变成“类型与状态的组合”。电气/open、电气/pending、电气/completed 不相同，都会保留；不能理解为“只按第一列去重”。它回答“有哪些不同组合”，不是“每种有多少条”，也不是修复错误查询的通用按钮。

想亲自核对时，用一个不会留下数据库文件的实验

前面的结果可以直接从六条记录推出来。实际运行时，在自己选择的实验目录建立 UTF-8 纯文本文件 `query_lab.py`，放入下面程序。它使用 Python 标准库，不需要安装数据库服务器。

`connect(':memory:')` 新建仅在这次连接里存在的数据库；`executescript` 执行给定的建表、填数脚本。这部分只是准备本章那六条已知事实，如何选择约束到 C03 再展开。真正需要替换、观察的是后半的 `sql` 字符串。

```python
import sqlite3

db = sqlite3.connect(":memory:")
```

PYTHON · 续

```
try:
    db.executescript("""
        CREATE TABLE appointments (
            id INTEGER PRIMARY KEY,
            customer_name TEXT,
            service_type TEXT,
            status TEXT,
            starts_at TEXT,
            duration_minutes INTEGER,
            note TEXT
        );
        INSERT INTO appointments VALUES
        (101, '林青', '电气', 'open', '2026-08-22 09:00', 30, NULL),
        (102, '周宁', '水路', 'open', '2026-08-22 13:30', 60, '带配件'),
        (103, '陈晓', '电气', 'pending', '2026-08-23 08:30', NULL, NULL),
        (104, '吴凡', '门锁', 'open', '2026-08-21 16:00', 45, '已电话确认'),
        (105, '赵敏', '电气', 'completed', '2026-08-24 10:00', 30, '完成'),
        (106, '何川', '水路', 'open', '2026-08-22 11:00', NULL, '需确认');
    """)
    sql = """
        SELECT id, starts_at, customer_name
        FROM appointments
        WHERE starts_at >= '2026-08-22 00:00'
          AND status != 'completed'
        ORDER BY starts_at, id
        LIMIT 3;
    """
    for row in db.execute(sql):
        print(row)
finally:
    db.close()
```

在这个文件所在目录的 PowerShell 中运行：

POWERSHELL

```
python .\query_lab.py
```

每个打印出来的 Python 元组对应一条结果行，依次为 101、106、102。`execute` 把字符串里的 SQL 交给 SQLite，循环取得结果，`finally` 关闭连接；再次运行会从头建立相同六行，不继承上次修改。

可把 sql 内容换为本章的空值或聚合查询，先写下预期，再比较实际行。若得到零行，先看条件是否把未知值排掉；若行数对、顺序不对，先看排序；若数字看似合理，先问它统计的是所有行还是已知值。不要只用“没有报错”判定查询正确。

本章的新关系是“事实 → 条件 → 结果形状”。下一章的问题是：客户姓名在客户表，预约在预约表，答案需要两边的信息时，行应当怎样对应，为什么会变多或消失？

C02 | 多张表怎样组成一个答案，又为什么多了或少了行？

先确认两张表各自代表什么，再看一次连接会保留、丢失或放大哪些行。

排班员想知道：具备电气技能的技术员，下周各有多少预约？没有预约的合格技术员也必须出现，否则恰好有空的人会从结果中消失。

姓名、技能和预约不在同一张表。本章先找到符合技能要求的人，再接上他们的预约，最后把预约明细折叠成“每人一个数量”。问题始终是同一个：**结果的一行，现在代表谁？**

先把分散的事实摆在一起

这是一个新的小型查询实验，不是在修改 C01 的六行表。为看清表间关系，预约通过编号指向客户和技术员；本章不要求先建立 C03 的完整业务库。

`customers` 保存客户：

id	name
C1	林青
C2	吴凡

`technicians` 保存技术员：

id	name
T1	王玥
T2	陈敏
T3	周宁

`appointments` 保存预约：

id	customer_id	technician_id	starts_at
A1	C1	T1	2026-08-24 09:00
A2	C2	T1	2026-08-26 14:00
A3	C1	T3	2026-08-25 10:00
A4	C2	T1	2026-08-31 09:00

`customer_id=C1` 不是客户姓名，而是指向 `customers` 中 `id=C1` 的记录。客户改名时，预约仍然指向同一个客户。主键（primary key）唯一标识本表的一行；外键（foreign key）保存另一张表中对应键的值。数据库怎样拒绝不存在的引用，会在 C03 真正建表时展开。

客户 C1 对应 A1、A3 两条预约，这是一对多：一个客户可以有多条预约，本例每条预约只属于一个客户。表名相近不会使它们自动关联；对应条件必须写出来。

技能还有一层关系。`skills` 列出技能本身：

id	code
S1	electrical
S2	plumbing

`technician_skills` 每行表示“某位技术员具有某项技能”：

technician_id	skill_id
T1	S1
T1	S2
T2	S1
T3	S2

T1 有两项技能，S1 又对应两位技术员。这是多对多，需要中间表保存每一对关系，而不是给 `technicians` 不断增加 `skill1`、`skill2` 列。

客户引用

appointments 的多条预约
↓
customers 的一个客户

技术员引用

appointments 的多条预约
↓
一个技术员

技能关系中的技术员引用

technician_skills 的每一对关系
↓
一个技术员

技能关系中的技能引用

technician_skills 的每一对关系
↓
skills 的一项技能

箭头表示“某行指向哪条事实”，不是运行指令。这里所有关系都已经能在小表中逐行找到。

先只找人：为什么可以把一条查询放进另一条？

暂时不算预约，先找具有 electrical 技能的技术员。第一步由技能代码找到 S1，再找中间表里的 T1、T2：

```sql
SELECT technician_id
FROM technician_skills
WHERE skill_id IN (
    SELECT id FROM skills WHERE code = 'electrical'
)
ORDER BY technician_id;
```

括号里的查询先给出一组可用于判断的值，当前只有 `S1`。外层的 `IN` 问“这一行的 skill_id 是否属于这组值”，所以留下 T1、T2。

想取得姓名，再以这组技术员编号筛选 `technicians`：

```sql
SELECT id, name
FROM technicians
WHERE id IN (
    SELECT technician_id
    FROM technician_skills
    WHERE skill_id IN (
        SELECT id FROM skills WHERE code = 'electrical'
    )
)
ORDER BY id;
```

结果为 T1/王玥、T2/陈敏。嵌在另一条查询中的查询叫子查询（subquery）。这里逐层产生编号集合，是解释逻辑的方式，不是断言引擎一定创建三张临时表。

如果只想问“这个人是否属于合格人选”，子查询配 `IN` 很直接。若结果要同时显示两边的列，例如客户姓名与预约时间，就需要把相应行组合到一起。

JOIN 怎样让客户姓名出现在预约旁边？

```sql
SELECT c.name AS customer_name,
       a.id AS appointment_id,
       a.starts_at
FROM customers AS c
JOIN appointments AS a ON a.customer_id = c.id
ORDER BY a.id;
```

`AS c`、`AS a` 给表起本条查询内的短名字。`c.id` 是客户表的编号，`a.id` 是预约表的编号；两边都有 `id` 时，写清来源才不会混淆。

`JOIN ... ON ...` 的意思是：把满足 `ON` 对应条件的左右行组合起来。这里的 `JOIN` 指内连接（inner join），只保留能对应上的组合。

customer_name	appointment_id	starts_at
林青	A1	2026-08-24 09:00
吴凡	A2	2026-08-26 14:00
林青	A3	2026-08-25 10:00
吴凡	A4	2026-08-31 09:00

林青出现两次，是因为 C1 分别与 A1、A3 对应。结果的一行代表一条预约及其客户信息，不代表一个独立客户；这不是错误重复。

反过来，若漏写对应条件，把两位客户分别和四条预约任意配对，会产生八行，甚至把林青配到吴凡的预约上。此时加 `DISTINCT` 也不能恢复谁属于谁。必须先修复关系条件。

外键和 `JOIN` 也不是同一件事：外键用于限制可保存的引用，`JOIN` 用于取得本次查询需要的组合。即使一个练习没有声明外键，只要有正确列和值，查询也能按条件连接；但它不再因此获得“坏引用不能写入”的保护。

为什么有空的技术员反而不见了？

回到排班问题。本章“下周”特指 `2026-08-24 00:00` 到 `2026-08-31 00:00` 之前，包含开始、不包含结束。A1、A2、A3 在这段时间内，A4 不在。

先取得电气技术员的预约明细：

SQL

```
SELECT t.id AS technician_id, t.name,
       a.id AS appointment_id
FROM technicians AS t
JOIN technician_skills AS ts ON ts.technician_id = t.id
JOIN skills AS s ON s.id = ts.skill_id
JOIN appointments AS a ON a.technician_id = t.id
WHERE s.code = 'electrical'
  AND a.starts_at >= '2026-08-24 00:00'
  AND a.starts_at < '2026-08-31 00:00'
ORDER BY t.id, a.id;
```

沿前三张表，electrical 找到 T1、T2；连接预约后，T1 找到 A1、A2、A4，时间条件排掉 A4。T2 没有任何预约可配，所以结果只有 T1/A1、T1/A2。

这条查询适合“列出已有预约”，却不满足“没有预约也要显示”。要保留左侧已合格的人，需要左连接（left join）：右侧找不到匹配时仍留一行，把右侧列补为 NULL。

```SQL
SELECT t.id AS technician_id, t.name,
       a.id AS appointment_id
FROM technicians AS t
JOIN technician_skills AS ts ON ts.technician_id = t.id
JOIN skills AS s ON s.id = ts.skill_id
LEFT JOIN appointments AS a
  ON a.technician_id = t.id
 AND a.starts_at >= '2026-08-24 00:00'
 AND a.starts_at < '2026-08-31 00:00'
WHERE s.code = 'electrical'
ORDER BY t.id, a.id;
```

technician_id	name	appointment_id
T1	王玥	A1
T1	王玥	A2
T2	陈敏	NULL

此时的 NULL 不是某条预约忘了填写编号，而是左连接为“没有匹配预约”补出来的空位置。

注意时间条件放在 ON 里：它限定哪些预约能与技术员匹配。若把右表的时间条件移回 WHERE，T2 那行的 a.starts_at 是 NULL，比较得不到真，这行又会被筛掉。

LEFT JOIN 先保留未匹配的左行，不保证后面的 WHERE 还会保留它。 先区分“什么可以配上”和“配好以后保留哪些结果行”，比只记连接名字可靠。

怎样从两条预约，得到“王玥有两条”？

现在的三行仍是明细。排班员要的是每位技术员一行，因此应先按技术员划分组，再数组内的预约编号：

```SQL
SELECT t.id AS technician_id, t.name,
       COUNT(a.id) AS appointment_count
FROM technicians AS t
JOIN technician_skills AS ts ON ts.technician_id = t.id
JOIN skills AS s ON s.id = ts.skill_id
LEFT JOIN appointments AS a
  ON a.technician_id = t.id
 AND a.starts_at >= '2026-08-24 00:00'
 AND a.starts_at < '2026-08-31 00:00'
WHERE s.code = 'electrical'
GROUP BY t.id, t.name
ORDER BY t.id;
```

`GROUP BY t.id, t.name` 将相同技术员的结果行归为一组。组内的 `COUNT(a.id)` 只数非 `NULL` 的预约编号：

T1 组：A1、A2 → 2
T2 组：NULL → 0

technician_id	name	appointment_count
T1	王玥	2
T2	陈敏	0

如果写 `COUNT(*)`，T2 会变成 1，因为它把左连接保留的占位行也数进去了。要数的是预约，所以选择在真实预约中不会为空的 `a.id`。按姓名独自分组也有风险：两位同名技术员可能被合成一组；这里把身份编号一起写明。

如果后续只需要下周预约至少两条的人，可以在 `GROUP BY` 之后、`ORDER BY` 之前加入：

```SQL
HAVING COUNT(a.id) >= 2
```

这是一段插入原查询的子句，不是可独立执行的 SQL。结果只剩 T1/王玥/2。`WHERE` 筛参与分组的输入行，`HAVING` 筛已经形成的组；“哪些预约算进来”和“统计后哪些人达标”不是同一步。

到这里可以把一处常见混淆说清楚：要每位技术员一行，为什么不直接给明细加 DISTINCT？如果保留预约编号，A1 与 A2 本就不同，不会合并；如果只保留技术员列，的确能列出不同的人，却没有算出预约数。应该先确定答案是一份人选名单、一张预约明细还是一组统计，再决定工具。

多对多连接为什么也会放大数量？

上面的排班查询只选 electrical，因此 T1 在技能这一侧只保留一对关系。如果去掉技能限制，同时接上 T1 的两项技能与两条周内预约，会得到四个组合：

skill_id	appointment_id
S1	A1
S1	A2
S2	A1
S2	A2

此时直接 COUNT(a.id) 得到 4，不等于真实预约数 2。每一行代表“技术员的一项技能与一条预约的组合”。表中没有新增预约，是查询把两个一对多分支交叉组合了。

若只需确认技术员具有目标技能，前面的子查询 IN 可以先筛人，不把每项技能明细带进后续统计。若目标确实是不同预约编号的数量，COUNT(DISTINCT a.id) 也能明确表达这个口径；但它不能为错误的 ON 条件作担保。先找出是哪一层增加了组合，再决定如何改写。

两份名单要合并或求交集时，还是 JOIN 吗？

“会电气的人”和“会水路的人”可以各自产生一列技术员编号。现在要做的是名单之间的集合操作，而不是把两张表的列并排接在一行：

```sql
SELECT technician_id
FROM technician_skills WHERE skill_id = 'S1'
UNION
SELECT technician_id
FROM technician_skills WHERE skill_id = 'S2'
ORDER BY technician_id;
```

UNION 合并两边并去除重复，得到 T1、T2、T3。把中间的 UNION 换成 INTERSECT，只留两边都有的 T1；换成 EXCEPT，取会电气但不在水路名单中的 T2。UNION ALL 则保留两边原有重复，不表示“独立人数”。

两边应当交出数量一致、业务含义可比较的列；不能把技术员编号和预约时间凑成“一个集合”。JOIN **根据关系组合列与行；这些集合操作组合的是两份同形状答案。** 后者需要时再查关键字，先能辨别自己是在关联事实还是合并名单。

把这些已知行交给 SQLite 核对

在独立实验目录建立 UTF-8 文件 relation_lab.py。下面先提供完整夹具，再运行本章的最终排班查询。建表中的主键、外键用于把已经说明的对应关系写出来；完整的约束设计不需要在这里提前展开。

PYTHON

```
import sqlite3

db = sqlite3.connect(":memory:")
try:
    db.execute("PRAGMA foreign_keys = ON")
    db.executescript("""
        CREATE TABLE customers (id TEXT PRIMARY KEY, name TEXT);
        CREATE TABLE technicians (id TEXT PRIMARY KEY, name TEXT);
        CREATE TABLE skills (id TEXT PRIMARY KEY, code TEXT UNIQUE);
        CREATE TABLE technician_skills (
            technician_id TEXT REFERENCES technicians(id),
            skill_id TEXT REFERENCES skills(id),
            PRIMARY KEY (technician_id, skill_id)
        );
        CREATE TABLE appointments (
            id TEXT PRIMARY KEY,
            customer_id TEXT REFERENCES customers(id),
            technician_id TEXT REFERENCES technicians(id),
            starts_at TEXT
        );
        INSERT INTO customers VALUES ('C1', '林青'), ('C2', '吴凡');
        INSERT INTO technicians VALUES
            ('T1', '王玥'), ('T2', '陈敏'), ('T3', '周宁');
        INSERT INTO skills VALUES ('S1', 'electrical'), ('S2', 'plumbing');
        INSERT INTO technician_skills VALUES
            ('T1', 'S1'), ('T1', 'S2'), ('T2', 'S1'), ('T3', 'S2');
        INSERT INTO appointments VALUES
            ('A1', 'C1', 'T1', '2026-08-24 09:00'),
            ('A2', 'C2', 'T1', '2026-08-26 14:00'),
            ('A3', 'C1', 'T3', '2026-08-25 10:00'),
            ('A4', 'C2', 'T1', '2026-08-31 09:00');
    """)
    sql = """
        SELECT t.id AS technician_id, t.name,
               COUNT(a.id) AS appointment_count
        FROM technicians AS t
```

PYTHON · 续

```
        JOIN technician_skills AS ts ON ts.technician_id = t.id
        JOIN skills AS s ON s.id = ts.skill_id
        LEFT JOIN appointments AS a
          ON a.technician_id = t.id
         AND a.starts_at >= '2026-08-24 00:00'
         AND a.starts_at < '2026-08-31 00:00'
        WHERE s.code = 'electrical'
        GROUP BY t.id, t.name
        ORDER BY t.id;
    """
    for row in db.execute(sql):
        print(row)
finally:
    db.close()
```

`PRAGMA foreign_keys = ON` 为本次连接开启外键检查；它是 SQLite 的连接设置，不是查询里自动出现的步骤。程序只创建内存数据库，关闭连接后清除。

POWERSHELL

```
python .\relation_lab.py
```

输出是 `('T1', '王玥', 2)`、`('T2', '陈敏', 0)`。替换 `sql` 时可以比较内连接与左连接的明细，观察 T2 究竟在哪一步消失；也可以把 `COUNT(a.id)` 改成 `COUNT(*)`，核对错误的 1 来自哪一行。

本章把单表查询扩成了“关联 → 行的数量变化 → 结果含义”。下一章反过来问：这些对象和关系既然要长期保存，一开始应怎样建表，才能让不存在的客户、负时长和重复占用被明确拒绝？

C03 | 怎样让坏数据进不来

先追一条预约从输入到约束拒绝的路线，再看规则怎样写进表。

维修预约里写着“客户 C999 的工单”，客户表却没有 C999。这样的数据即使能被查询出来，也无法告诉工作人员应该联系谁。现在要解决的问题已经从“怎样读出正确答案”，走到“怎样让保存下来的事实本身可信”。

这一章从一张工单开始，先决定每个事实放在哪里，再把几种确定的坏值交给数据库拒绝。最后看看：表里已经有数据时，新规则怎样进入，才不会把旧事实丢在半路。

一张预约究竟连接了哪些事实？

林青报告“配电箱异响”，系统给问题编号 `T-42`；王玥提供一个 60 分钟的维修时段 `SL1`。预约 `A1` 的含义是：让这张工单使用这个时段。这里的 `T-42` 是本章独立数据库中的工单示例，不与其他程序共享记录。

先看准备保存的行。括号里的值按右侧列顺序排列：

表	一条实际记录	列与一行含义
customers	`(C1, 林青, lin@example.test, NULL)`	id、name、email、phone；一位客户
tickets	`(T-42, C1, 配电箱异响, 2, open, 1)`	id、customer_id、title、priority、status、version；一个待处理问题
technicians	`(T1, 王玥)`	id、name；一位技术员
slots	`(SL1, T1, 2026-09-01 09:00, 60, available)`	id、technician_id、starts_at、duration_minutes、status；一个可安排的时间段
appointments	`(A1, T-42, SL1, confirmed, NULL)`	id、ticket_id、slot_id、status、note；工单与时段的一次预约关系

appointments 不重复保存林青的邮箱。它保存 ticket_id='T-42'；找到工单后，再沿 customer_id='C1' 找到客户。技术员也从时段寻找：A1 → SL1 → T1。所以一张预约可以恢复完整的服务对象和安排：

预约沿工单找到客户

appointments：A1
↓
tickets：T-42
↓
customers：C1

预约沿时段找到技术员

appointments：A1
↓
slots：SL1
↓
technicians：T1

箭头从保存引用的行指向被引用的行。例如 tickets.customer_id → customers.id。一位客户可以有多张工单，一张工单也可以留下多条预约历史；一位技术员可以提供多个时段。本例先采用一个限制：一个时段最多出现在一条预约记录中。

这比“预约直接存客户 ID、技术员 ID”的查询小表多了两段关系。原因是现在要让工单和时段也拥有独立身份。工单还没有预约时可以存在，时段还没有被选择时也可以存在。这是从业务含义重新设计结构，不能把查询小表直接当成已经完成了迁移的数据库。

数据库怎样发现 C999 根本不存在？

先只建客户与工单之间的最小关系。在独立的 SQLite 小实验中执行：

```sql
PRAGMA foreign_keys = ON;
CREATE TABLE customer_demo (
    id TEXT PRIMARY KEY NOT NULL
);
CREATE TABLE ticket_demo (
    id TEXT PRIMARY KEY NOT NULL,
    customer_id TEXT NOT NULL REFERENCES customer_demo(id)
);
INSERT INTO customer_demo (id) VALUES ('C1');
INSERT INTO ticket_demo (id, customer_id) VALUES ('T-42', 'C1');
```

最后一行能写入，因为 customer_demo 确实存在 C1。把输入改成另一张工单：

```sql
INSERT INTO ticket_demo (id, customer_id) VALUES ('T-BAD', 'C999');
```

这次会得到外键约束错误，T-BAD 不会留下。数据库检查的不是文字是否以 C 开头，而是引用的目标行是否存在。FOREIGN KEY 就是把这条关系变成可以执行的规则。

`PRIMARY KEY` 负责让一个 ID 唯一找到当前表的一行；外键借用那个身份连接另一张表。姓名不适合作为这里的身份：两个客户可以都叫林青，客户改名也不应该让所有工单失去联系。

SQLite 还需要在每个连接上实际开启外键检查。`PRAGMA foreign_keys;` 应返回 `1`；开启语句应放在事务开始之前，事务中切换可能无效。建表文字说明“应该怎样”，连接设置决定当前检查是否启用。这个行为以 SQLite 的外键文档为准。

一列写着 INTEGER，为什么还能收下文字？

时段长度应该是正整数分钟。先不要急着认为 `INTEGER` 已经负责全部检查，运行一个只有类型声明的小表：

```sql
CREATE TABLE affinity_demo (minutes INTEGER);
INSERT INTO affinity_demo (minutes)
VALUES ('60'), ('thirty'), (3.5), (-30), (NULL);
SELECT minutes, typeof(minutes) FROM affinity_demo ORDER BY rowid;
```

它会留下五行：

minutes	typeof(minutes)
60	integer
thirty	text
3.5	real
-30	integer
NULL	null

普通 SQLite 表使用类型亲和性（type affinity）：列偏好某种存储类别，并尝试适当转换。`'60'` 可以变成整数；`'thirty'` 转不了，仍然存成文字。`-30` 本来就是整数，类型也不知道维修时长不能为负。SQLite 的存储类别还有用于小数的 REAL、文字 TEXT、二进制 BLOB，以及表示空值的 NULL。类型声明和业务允许范围是两件事。SQLite 类型文档

因此时长列需要把要求补全：

```sql
duration_minutes INTEGER NOT NULL
    CHECK (typeof(duration_minutes) = 'integer'
           AND duration_minutes > 0)
```

NOT NULL 拒绝未知时长，CHECK 要求实际存下来的值是整数且大于零。'60' 经转换后仍能通过，因为这条规则针对存储结果；它并不证明输入最初就是 Python 整数。

类型表达存储意图，约束表达必须成立的规则。 面对一个坏值，应该能指出是“缺失”“种类不对”，还是“范围不对”，再找到负责拒绝它的位置。

哪些事实必须有值，哪些未知应该留下？

新工单从 open 开始，第一版编号为 1。省略这两列时，业务本来就能确定值，所以可以使用 DEFAULT 'open' 和 DEFAULT 1。

电话不同。林青尚未提供电话，这时保存 NULL 才符合事实。'' 是长度为零的文字，'0000000' 则是一个编造出来的值；后续通知程序若把它当作号码，会把“未知”误读成“已经可以联系”。

本例允许电话未知；提供电话时，至少要有七个非首尾空格字符：

```SQL
phone TEXT CHECK (phone IS NULL OR length(trim(phone)) >= 7)
```

trim 去掉首尾空格，length 计算长度。这是很有限的格式规则：能拒绝空文字，但不能证明电话属于本人，也不能证明能拨通。后两件事要靠应用的验证流程。

CHECK 还有一个容易漏掉的细节：表达式结果为 NULL 时不算违反检查。因此，若状态一定要有值，单写 CHECK (status IN (...)) 不够，还需要 NOT NULL。普通 SQLite 表中的文字主键也显式加上 NOT NULL，避免历史兼容行为允许空主键；UNIQUE 本身同样不拒绝多个 NULL。SQLite 建表与约束文档

默认值负责“省略了，就采用确定的初值”。在本章普通 INSERT 中显式写 status=NULL，会被 NOT NULL 拒绝，不会变回 open。备注可以未知，就保留可空列；不能为了让所有格子都有内容而制造假事实。

技能为什么还需要一张关系表？

王玥会电气维修，也会水路维修。若把 electrical,plumbing 塞进 technicians.skills 一格，数据库没法直接用一个外键分别检查两项技能；查询某种技能的人也得先解析文字。

把技能本身与拥有技能的关系分开后，可以保存：

```
skills:              (S1, electrical, 电气)
                     (S2, plumbing, 水路)
technician_skills:   (T1, S1)
                     (T1, S2)
```

技能 S1 可以被多位技术员拥有，技术员 T1 也可以拥有多种技能。中间表的一行只说“这个人拥有这个技能”；组合主键 (technician_id, skill_id) 让 (T1,S1) 不能重复登记，但允许 (T1,S2) 或 (T2,S1)。

现在“电气”改一个名称，只改 skills 的一行。客户邮箱改了，也只改 customers 的一行；工单仍然通过 C1 找到最新值。这种减少同一事实重复保存、避免修改遗漏的设计，就是这里要用到的规范化（normalization）。**一个事实有明确的保存位置，其他地方通过身份找到它。**

这不要求把标题的每个词或姓名的每个字拆成表。是否拆分取决于它能不能独立变化、能不能拥有独立关系，以及重复保存会不会产生矛盾。如果要保留“当时发信所用地址”，那是另一种历史事实，不能与“客户现在的邮箱”混为一谈。

把这些决定放在一起，表应该怎样建？

下面的完整结构称为 schema。它延续刚才五种对象，再补上技能与技能关系。为便于接下来的实验，把这一整个 SQL 块保存为自选实验目录中的 c_schema.sql。这是完整建表内容，前面的 *_demo 小表不需要复制进去。

SQL

```
PRAGMA foreign_keys = ON;

CREATE TABLE customers (
    id TEXT PRIMARY KEY NOT NULL,
    name TEXT NOT NULL CHECK (length(trim(name)) >= 2),
    email TEXT NOT NULL UNIQUE,
    phone TEXT CHECK (phone IS NULL OR length(trim(phone)) >= 7)
);
CREATE TABLE technicians (
    id TEXT PRIMARY KEY NOT NULL,
    name TEXT NOT NULL CHECK (length(trim(name)) >= 2)
);
CREATE TABLE skills (
    id TEXT PRIMARY KEY NOT NULL,
    code TEXT NOT NULL UNIQUE,
    label TEXT NOT NULL UNIQUE
);
CREATE TABLE technician_skills (
    technician_id TEXT NOT NULL,
    skill_id TEXT NOT NULL,
    PRIMARY KEY (technician_id, skill_id),
    FOREIGN KEY (technician_id) REFERENCES technicians(id) ON DELETE RESTRICT,
    FOREIGN KEY (skill_id) REFERENCES skills(id) ON DELETE RESTRICT
);
CREATE TABLE tickets (
    id TEXT PRIMARY KEY NOT NULL,
```

SQL · 续

```
    customer_id TEXT NOT NULL,
    title TEXT NOT NULL CHECK (length(trim(title)) >= 3),
    priority INTEGER NOT NULL
        CHECK (typeof(priority) = 'integer' AND priority BETWEEN 1 AND 3),
    status TEXT NOT NULL DEFAULT 'open'
        CHECK (status IN ('open', 'pending', 'closed')),
    version INTEGER NOT NULL DEFAULT 1
        CHECK (typeof(version) = 'integer' AND version >= 1),
    FOREIGN KEY (customer_id) REFERENCES customers(id) ON DELETE RESTRICT
);
CREATE TABLE slots (
    id TEXT PRIMARY KEY NOT NULL,
    technician_id TEXT NOT NULL,
    starts_at TEXT NOT NULL,
    duration_minutes INTEGER NOT NULL
        CHECK (typeof(duration_minutes) = 'integer' AND duration_minutes > 0),
    status TEXT NOT NULL DEFAULT 'available'
        CHECK (status IN ('available', 'held', 'booked')),
    FOREIGN KEY (technician_id) REFERENCES technicians(id) ON DELETE RESTRICT
);
CREATE TABLE appointments (
    id TEXT PRIMARY KEY NOT NULL,
    ticket_id TEXT NOT NULL,
    slot_id TEXT NOT NULL UNIQUE,
    status TEXT NOT NULL DEFAULT 'confirmed'
        CHECK (status IN ('confirmed', 'cancelled')),
    note TEXT,
    FOREIGN KEY (ticket_id) REFERENCES tickets(id) ON DELETE RESTRICT,
    FOREIGN KEY (slot_id) REFERENCES slots(id) ON DELETE RESTRICT
);
```

这里的姓名最少两个字符、标题最少三个字符，只是这份练习明确采用的输入规则，不代表所有姓名和标题都适用。邮箱也采用一个已决定的规则：应用先规范化，再交给唯一约束；当前默认文字比较不会自动把大小写不同或带多余空格的邮箱视作同一个地址。

看 `appointments.slot_id`：外键证明 SL1 存在，`UNIQUE` 再证明它没有被另一条预约引用。两者合起来才形成“每条预约使用一个真实时段，一个时段最多被一条预约使用”的数量关系。这个唯一限制也包含取消的预约；以后若要求取消后重新预订同一时段，必须重新设计这条规则，不能只修改一个状态就假定完成了复用。

`ON DELETE RESTRICT` 则保护引用的另一端：还有工单引用 C1 时，不能删掉 C1。它的具体效果会在取消与删除时更明显。

这份 schema 仍不能检查所有事情。`starts_at TEXT NOT NULL` 没有证明日期存在、处在营业时间或没有重叠；技能关系表也没有自动证明这次安排满足工单所需技能。当前能由结构判断的规则由数据库保护，日期解析、身份权限、技能匹配等需要应用明确完成。

怎样知道约束真的在工作?

先准备基础事实，把下面 SQL 块保存为同一目录中的 `c_seed.sql`。种子数据只提供可预约的客户、工单、技术员与时段，尚未执行“确认预约”：

```sql
INSERT INTO customers (id, name, email, phone)
VALUES ('C1', '林青', 'lin@example.test', NULL);
INSERT INTO technicians (id, name) VALUES ('T1', '王玥');
INSERT INTO skills (id, code, label) VALUES ('S1', 'electrical', '电气');
INSERT INTO technician_skills (technician_id, skill_id) VALUES ('T1', 'S1');
INSERT INTO tickets (id, customer_id, title, priority)
VALUES ('T-42', 'C1', '配电箱异响', 2);
INSERT INTO slots (id, technician_id, starts_at, duration_minutes)
VALUES ('SL1', 'T1', '2026-09-01 09:00', 60),
       ('SL2', 'T1', '2026-09-01 10:30', 30);
```

可以用 Python 3.12 或更新版本的标准库在内存里运行它们。把下面程序保存为同目录的 `c03_check.py`，运行 `python c03_check.py`。数据库随程序关闭而结束，不接触现有业务文件。

```python
from pathlib import Path
import sqlite3

folder = Path(__file__).resolve().parent
connection = sqlite3.connect(':memory:', autocommit=True)
connection.executescript((folder / 'c_schema.sql').read_text(encoding='utf-8-
  sig'))
connection.executescript((folder / 'c_seed.sql').read_text(encoding='utf-8-
  sig'))
assert connection.execute('PRAGMA foreign_keys').fetchone()[0] == 1

connection.execute("""
    INSERT INTO appointments (id, ticket_id, slot_id)
    VALUES ('A1', 'T-42', 'SL1')
""")
print(connection.execute(
    "SELECT id, status, version FROM tickets WHERE id='T-42'"
).fetchone())
print(connection.execute(
    "SELECT id, status, note FROM appointments WHERE id='A1'"
).fetchone())

bad_rows = [
    ("INSERT INTO customers (id,name,email) VALUES
      (NULL,'周宁','n@example.test')", ()),
```

PYTHON · 续

```
    ("INSERT INTO customers (id,name,email) VALUES
      ('C2','周宁','lin@example.test')", ()),
    ("INSERT INTO slots (id,technician_id,starts_at,duration_minutes) VALUES
      ('BAD','T1','2026-09-02 09:00',?)", (-30,)),
    ("INSERT INTO slots (id,technician_id,starts_at,duration_minutes) VALUES
      ('BAD','T1','2026-09-02 09:00',?)", ('thirty',)),
    ("INSERT INTO tickets (id,customer_id,title,priority) VALUES
      ('T-BAD','C999','照明检查',2)", ()),
    ("INSERT INTO customers (id,name,email,phone) VALUES
      ('C3','周宁','z@example.test','')", ()),
    ("INSERT INTO appointments (id,ticket_id,slot_id) VALUES
      ('A2','T-42','SL1')", ()),
]
for sql, values in bad_rows:
    before = list(connection.iterdump())
    try:
        connection.execute(sql, values)
    except sqlite3.IntegrityError as error:
        print(error.sqlite_errorname)
    else:
        raise AssertionError('坏行没有被拒绝')
    assert list(connection.iterdump()) == before

connection.close()
```

前两行输出是 `('T-42', 'open', 1)` 与 `('A1', 'confirmed', None)`；Python 的 `None` 对应 SQL 的 `NULL`。随后七次尝试分别被非空、唯一、检查、检查、外键、检查、唯一约束拒绝。比较前后数据，是为了确认不仅出现了错误，而且失败没有留下新行。

这里 `execute(sql, values)` 把 SQL 和输入值分开交给数据库，`?` 依次接受元组里的值；下一章继续用这一个入口写入，应用访问一章再解释为什么不能拼接文字。`iterdump()` 只用于这份极小合成库的前后比较，不是日常日志，也不要打印真实客户数据库的全部内容。

现在 A1 引用正确，却没有同步把 SL1 改成 `booked`。这个实验只证明结构允许与拒绝哪些行，还没有证明完整预约动作成功。单条约束无法凭空补齐另一个表的状态变化，这正是下一章事务要解决的问题。

已经存了旧数据，新规则怎么加进去？

修改 `CREATE TABLE` 文件只会改变将来怎样建新库，不会更新已经存在的表。迁移（migration）要处理的是“旧结构和旧行怎样一起到达新结构”。

简单的例子是增加可空的联系偏好。下面 SQL 在本章临时库中单独运行：

```sql
ALTER TABLE customers ADD COLUMN preferred_contact TEXT
    CHECK (preferred_contact IS NULL OR preferred_contact IN ('email',
      'phone'));
SELECT id, preferred_contact FROM customers;
```

C1 的新列为 NULL，因为旧记录从未提供这项偏好。不能擅自当成 email。这个小实验不是下一章核心 schema 的一部分，重新运行种子程序仍从原先七张表开始。

如果旧预约表根本没有状态检查，事情更复杂。先用两行旧数据观察问题：

```sql
CREATE TABLE legacy_appointments (id TEXT PRIMARY KEY NOT NULL, status TEXT);
INSERT INTO legacy_appointments VALUES ('OLD1','confirmed'),
  ('OLD2','confirmd');
SELECT id, status FROM legacy_appointments
WHERE status IS NULL OR status NOT IN ('confirmed','cancelled');
```

结果指出 OLD2, confirmd。新规则要求 confirmed/cancelled，不能让复制过程静默丢掉 OLD2，也不能未核实就把它解释为取消。

下面只演练复制与撤回，不替换任何正式表。BEGIN 先圈出一组尚未提交的改变；ROLLBACK 撤回这组改变。它们的完整工作方式会在下一章追踪。

```sql
BEGIN;
CREATE TABLE checked_appointments (
    id TEXT PRIMARY KEY NOT NULL,
    status TEXT NOT NULL CHECK (status IN ('confirmed','cancelled'))
);
INSERT INTO checked_appointments (id,status)
SELECT id,status FROM legacy_appointments;
```

复制会被 OLD2 的状态拒绝。看到错误后，明确执行：

```sql
ROLLBACK;
SELECT id,status FROM legacy_appointments ORDER BY id;
SELECT name FROM sqlite_schema WHERE name='checked_appointments';
```

旧表仍有原来的两行，新表不存在。只出现一条约束错误，并不等于整个迁移已经自动撤销，所以恢复动作和恢复后读取都不能省略。

若来源确认 OLD2 是拼写错误，可以在这份实验副本中改成 confirmed，再重新执行同一段复制，核对新旧每一行。即使这次复制成功，仍执行 `ROLLBACK`，证明成功的演练也能退出，不把“复制成功”说成“正式切换完成”。

真正切换被其他表、view 和应用引用的表时，还要处理这些依赖、核对完整行与外键，并准备可恢复副本。SQLite 对增加 CHECK 等变化给出了重建表的通用流程；应按照目标版本的ALTER TABLE 文档在副本演练，不能把这份两列小实验当成完整业务迁移脚本。

到这里值得能够自己预测的是：C999 会在外键处失败，负数会在时长检查处失败，未知电话应保持 NULL；给旧表加规则时，旧行也必须接受同样的检查。完整建表与迁移步骤可以查回，但这些因果关系决定查什么、怎样判断结果，不能只靠记住约束名字。

本章对象、关系、规范化与设计问题对应 CS50 SQL Week 2。本章引用的 SQLite 与 Python 操作文档是持续更新的 live 文档；实际运行时应核对目标版本。严格类型表、服务器数据库差异以及复杂迁移，只在当前项目需要它们时再展开，不承担这里七表实验的前提。

C04 | 几步修改怎样一起成功或撤回

先追一笔确认同时改变的三处事实，再看事务怎样让它们全成或全撤。

林青确认了维修预约。数据库新增 A1，却没有把时段 SL1 改为已占用；另一位工作人员看到它仍然可用，又准备安排一次服务。每张表里的单个值都可能合法，整件事却没有完成。

确认预约需要同时留下三种事实：预约存在、时段被占用、这次确认有一条持久事件。本章沿这三处变化解释事务，再把相同的完整性要求用到批量导入与取消操作中。

一次“确认成功”究竟要求哪几处同时改变？

先恢复本章用到的事实。客户 `C1` 是林青；工单 `T-42` 属于 C1，标题是“配电箱异响”。技术员 `T1` 是王玥；时段 `SL1` 属于 T1，开始时间为 `2026-09-01 09:00`，时长 60 分钟。另一个时段 SL2 在同日 10:30 开始，时长 30 分钟。两个时段起初都为 `available`。

预约通过工单找到客户，通过时段找到技术员：

预约沿工单找到客户

appointments
↓
tickets
↓
customers

预约沿时段找到技术员

appointments
↓
slots
↓
technicians

事件引用预约

appointment_events → appointments

这里的箭头同样从引用列指向被引用对象。七张业务表沿用 C03 的结构，核心模型只增加 `appointment_events`；技能与技术员的关系仍由 `skills`、`technician_skills` 保存。

事件表的一行说明“哪次处理对哪个预约做了什么”。`request_id='R-42'` 用来关联这一次处理；它不是工单 T-42，也不是预约 A1。事件并未保存操作者身份，因此仅凭它不能回答“具体哪个人确认了预约”。

```sql
CREATE TABLE appointment_events (
    id INTEGER PRIMARY KEY,
    appointment_id TEXT NOT NULL,
    request_id TEXT NOT NULL,
    event_type TEXT NOT NULL
        CHECK (event_type IN ('appointment_confirmed',
          'appointment_cancelled')),
    FOREIGN KEY (appointment_id) REFERENCES appointments(id) ON DELETE RESTRICT
);
```

事件 ID 是数据库生成的整数身份；调用者省略 id 即可。它和 request ID 的职责不同：一次批量处理可以写出多个事件，共用一个 request ID。

位置	确认前	完整确认后
appointments	没有 A1	A1 引用 T-42、SL1，状态 confirmed
slots	SL1 为 available	SL1 为 booked
appointment_events	没有 A1 的事件	一条 appointment_confirmed，关联 R-42

表中右侧三个结果共同构成成功。只出现其中一个或两个，就是需要避免的半成品。

为什么三条合法的 SQL 还不够?

`INSERT` 新增行，`UPDATE` 改变已有行，`DELETE` 移除行。先只看确认时的三条写入：

```sql
INSERT INTO appointments (id, ticket_id, slot_id)
VALUES ('A1', 'T-42', 'SL1');

UPDATE slots SET status='booked'
WHERE id='SL1' AND status='available';

INSERT INTO appointment_events (appointment_id, request_id, event_type)
VALUES ('A1', 'R-42', 'appointment_confirmed');
```

第一条使用预约的默认状态 confirmed。第二条用 `SET` 指明新值，用 `WHERE` 指明哪些旧行有资格变化。第三条留下事件。

如果每条都各自提交，第三条失败时，前两条可能已经成为长期状态。约束能拒绝非法事件，却不会替应用猜测“另外两条也属于同一件事”。必须先明确划出整件事的边界：

```
BEGIN
  新增预约
  占用时段
  新增事件
COMMIT
```

`BEGIN` 开始事务（transaction），把随后几条语句放入同一组尚未提交的改变；`COMMIT` 提交整组；`ROLLBACK` 撤回整组。**一条语句已经执行，不等于整件事已经提交。**

在事务内部，当前连接可以读到刚刚插入的 A1，用它继续写事件。提交成功后，新开启的后续读取才能以完整的新状态为起点；已经处于旧读取事务中的其他连接何时更新视图，是另一个隔离问题。

事件写坏时，预约为什么不会残留？

改用另一个空闲时段 SL2，尝试预约 A2。前三个时刻是：

执行位置	当前事务内 A2	SL2 状态	A2 事件数
BEGIN 后	不存在	available	0
INSERT 预约后	暂时存在	available	0
UPDATE 时段后	暂时存在	booked	0

然后故意把事件类型写成 `not_allowed`。事件表只允许两种类型，`CHECK` 拒绝这一行。

此刻要格外区分“错误语句被拒绝”和“整组自动撤销”。本例默认的约束失败通常只撤回当前语句；前两条改变还可能留在尚未结束的事务里。如果捕获错误后反而提交，仍可能留下半成品。应用必须转入 `ROLLBACK`，回到开始前的 `A2 不存在 / SL2 available / 0` 条事件。SQLite 事务文档

约束负责发现事件类型不合法，事务负责让相关改变一起撤回。没有前者，错误事件可能被提交；没有后者，发现错误也不能保证其他改变一起消失。

怎样把这条关系写成能够运行的程序？

使用 Python 3.12 或更新版本，把下面程序保存为 `c04_transaction.py`。同目录放 C03 完整展示的 `c_schema.sql` 与 `c_seed.sql`：前者是七张业务表，后者是本章开头的 C1、T-42、T1、S1、SL1、SL2，尚无预约。若中断后回来，从这两段完整 SQL 重新保存即可，不要沿用已经插入过 A1 的实验数据库。

程序每次建立新的内存库，再只增加事件表。建表与基础数据发生在业务事务之前，确认过程中不用 `executescript()` 混入另一套提交规则。

```python
from pathlib import Path
import sqlite3

EVENT_SCHEMA = """
CREATE TABLE appointment_events (
    id INTEGER PRIMARY KEY,
    appointment_id TEXT NOT NULL,
    request_id TEXT NOT NULL,
    event_type TEXT NOT NULL
        CHECK (event_type IN ('appointment_confirmed',
          'appointment_cancelled')),
    FOREIGN KEY (appointment_id) REFERENCES appointments(id) ON DELETE RESTRICT
);
"""

def setup():
    folder = Path(__file__).resolve().parent
    connection = sqlite3.connect(':memory:', autocommit=True)
    connection.executescript((folder /
      'c_schema.sql').read_text(encoding='utf-8-sig'))
    connection.executescript((folder /
      'c_seed.sql').read_text(encoding='utf-8-sig'))
    connection.executescript(EVENT_SCHEMA)
    assert connection.execute('PRAGMA foreign_keys').fetchone()[0] == 1
    return connection

def write_confirmation(connection, appointment_id, ticket_id, slot_id,
                       request_id, event_type='appointment_confirmed'):
    # 本函数完成三步写入，事务由调用者统一管理。
    connection.execute("""
        INSERT INTO appointments (id, ticket_id, slot_id)
        VALUES (?, ?, ?)
    """, (appointment_id, ticket_id, slot_id))
    changed = connection.execute("""
        UPDATE slots SET status='booked'
        WHERE id=? AND status='available'
    """, (slot_id,))
    if changed.rowcount != 1:
        raise ValueError('slot_not_available')
    connection.execute("""
        INSERT INTO appointment_events (appointment_id, request_id, event_type)
        VALUES (?, ?, ?)
    """, (appointment_id, request_id, event_type))
```

PYTHON · 续

```
def confirm(connection, appointment_id, ticket_id, slot_id, request_id,
            event_type='appointment_confirmed'):
    connection.execute('BEGIN')
    try:
        write_confirmation(connection, appointment_id, ticket_id, slot_id,
                           request_id, event_type)
        connection.execute('COMMIT')
    except Exception:
        if connection.in_transaction:
            connection.execute('ROLLBACK')
        raise

def state(connection, slot_id):
    status = connection.execute(
        'SELECT status FROM slots WHERE id=?', (slot_id,)
    ).fetchone()[0]
    appointments = connection.execute(
        'SELECT id FROM appointments WHERE slot_id=? ORDER BY id', (slot_id,)
    ).fetchall()
    events = connection.execute("""
        SELECT COUNT(*) FROM appointment_events AS e
        JOIN appointments AS a ON a.id=e.appointment_id
        WHERE a.slot_id=?
    """, (slot_id,)).fetchone()[0]
    return status, appointments, events

def main():
    connection = setup()
    try:
        confirm(connection, 'A1', 'T-42', 'SL1', 'R-42')
        assert state(connection, 'SL1') == ('booked', [('A1',)], 1)
        try:
            confirm(connection, 'A2', 'T-42', 'SL2', 'R-FAIL', 'not_allowed')
        except sqlite3.IntegrityError as error:
            print(error.sqlite_errorname)
        else:
            raise AssertionError('预设失败没有发生')
        assert state(connection, 'SL2') == ('available', [], 0)
        print(state(connection, 'SL1'))
        print(state(connection, 'SL2'))
    finally:
        connection.close()

if __name__ == '__main__':
    main()
```

运行 `python c04_transaction.py`，输出应为：

```
SQLITE_CONSTRAINT_CHECK
('booked', [('A1',)], 1)
('available', [], 0)
```

先看控制去向：`confirm` 开始事务，调用三步写入，提交成功才正常返回；发生异常就撤回仍在进行的事务，再把原异常交回调用者。`COMMIT` 也放在 `try` 内，因为提交本身可能失败，不能把“三条写入都没报错”当成最终成功。

这里特意选择 `autocommit=True`，让事务完全由可见的 SQL 命令管理。这是教学程序的一种明确选择；在这个模式下，Python 的 `connection.commit()` 和 `connection.rollback()` 不起作用，因此代码使用 `execute('COMMIT')` 与 `execute('ROLLBACK')`。其他连接模式可以使用那两个方法，不能把不同模式的写法随意混合。Python sqlite3 事务控制

再看实际影响范围。时段根本不存在时，第一步插入预约就会被外键拒绝，不会执行后面的 `UPDATE`。时段存在但已是 held 等非 available 状态，而且尚未被其他预约引用时，第一步可以暂时插入；随后 `UPDATE` 命中 0 行，函数抛出 `slot_not_available`，把刚才的插入一起撤回。若时段已经被另一条预约引用，则更早由第一步的 `UNIQUE` 拒绝。三种失败都会进入撤回路径，但最早拒绝的位置不同。`?` 从后面的元组取得 slot_id，输入没有进入 SQL 结构。

写前可以用相同条件预览：`SELECT id,status FROM slots WHERE id='SL1' AND status='available';`。预览帮助确认目标，但两次操作之间状态可能变化，所以真正 UPDATE 仍保留条件并检查影响行数。这个主例验证单连接的指定成功和失败；它没有证明所有并发交错都安全，也没有保证多次重放自动只产生一次效果。

三行导入，第二行写着 thirty，第一行能先留下吗？

批量来源会带来还没有解释的文字。假设一份预约 CSV 包含三行，要求整批成功才接收：

行号	ticket_id	customer_id	slot_id	starts_at	duration_minutes
1	T-42	C1	SL1	2026-09-01 09:00	60
2	T-42	C1	SL2	2026-09-01 10:30	thirty
3	T-42	C1	SL3	2026-09-01 14:00	45

时段的时间与分钟数在正式 `slots` 中已经有权威值。CSV 重复这些字段，是为了验证来源指向了预期时段，不允许它顺便覆盖正式时长。客户 ID 也必须与工单实际所属客户相符。

先放入 staging table，即只保存待处理原始值的暂存表。所有业务字段先按文字留下，第二行的 `thirty` 才能原样定位；暂存表接住它，不代表正式预约已经接受它。

以下 SQL 从 `setup()` 建立的全新数据库开始，另加 SL3；不承接已经成功预约 A1 的那次运行：

```sql
INSERT INTO slots (id,technician_id,starts_at,duration_minutes)
VALUES ('SL3','T1','2026-09-01 14:00',45);
CREATE TEMP TABLE appointment_staging (
    line_number INTEGER PRIMARY KEY,
    ticket_id_text TEXT NOT NULL,
    customer_id_text TEXT NOT NULL,
    slot_id_text TEXT NOT NULL,
    starts_at_text TEXT NOT NULL,
    duration_text TEXT NOT NULL
);
INSERT INTO appointment_staging VALUES
    (1,'T-42','C1','SL1','2026-09-01 09:00','60'),
    (2,'T-42','C1','SL2','2026-09-01 10:30','thirty'),
    (3,'T-42','C1','SL3','2026-09-01 14:00','45');
```

下面函数接在 `c04_transaction.py` 的函数定义后面使用。它会解析分钟和日期，核对工单客户、时段内容与可用状态，最后才返回准备好的预约行。

```python
from datetime import datetime

def prepare_batch(connection):
    prepared, errors = [], []
    used_slots = set()
    rows = connection.execute(
        'SELECT * FROM appointment_staging ORDER BY line_number'
    ).fetchall()
    for line, ticket, customer, slot, starts_text, duration_text in rows:
        try:
            minutes = int(duration_text)
            if minutes <= 0:
                raise ValueError
        except ValueError:
            errors.append((line, 'duration_invalid'))
            continue
        try:
            starts = datetime.strptime(starts_text, '%Y-%m-%d %H:%M')
        except ValueError:
            errors.append((line, 'starts_at_invalid'))
            continue
```

PYTHON · 续

```
        known = connection.execute("""
            SELECT t.customer_id, s.starts_at, s.duration_minutes, s.status
            FROM tickets AS t CROSS JOIN slots AS s
            WHERE t.id=? AND s.id=?
        """, (ticket, slot)).fetchone()
        if known is None:
            errors.append((line, 'ticket_or_slot_missing'))
        elif (customer, starts.strftime('%Y-%m-%d %H:%M'), minutes,
              'available') != known:
            errors.append((line, 'source_does_not_match'))
        elif slot in used_slots:
            errors.append((line, 'duplicate_slot_in_batch'))
        else:
            used_slots.add(slot)
            prepared.append((f'A-B{line}', ticket, slot))
    if errors:
        raise ValueError(errors)
    return prepared

def import_batch(connection, request_id):
    connection.execute('BEGIN')
    try:
        prepared = prepare_batch(connection)
        for appointment_id, ticket_id, slot_id in prepared:
            write_confirmation(connection, appointment_id, ticket_id, slot_id,
                               request_id)
        connection.execute('COMMIT')
    except Exception:
        if connection.in_transaction:
            connection.execute('ROLLBACK')
        raise
```

`CROSS JOIN` 在这里组合指定的一张工单与一个时段，以便同时核对二者；两个 `WHERE` 条件都使用唯一 ID，最多得到一行。它没有建立新外键，也没有改变工单通过客户 ID、时段通过技术员 ID 的实际关系。

准备阶段发现第二行分钟数无法转换，抛出 `ValueError([(2, 'duration_invalid')])`。第 1、3 行即使合法，整个准备函数仍不交出可写入结果，所以正式预约和事件新增都是 0，三个时段都是 available。原始三行在事务开始前进入临时表，撤回本次导入后仍在当前连接中，方便定位错误。

核实来源后，把第二行 `duration_text` 改为 `'30'`，重新调用 `import_batch`。这次三行都通过验证，事务内写出 A-B1、A-B2、A-B3，把三个时段变为 booked，并写三条共用本次 request ID 的事件。如果正式写入期间任意一步失败，整批也撤回；事前检查通过不能代替写入时的约束。

这段实验可以另存为 `c04_batch.py`：先 `from c04_transaction import setup, write_confirmation`，复制上面的两个函数与 datetime 导入，将暂存 SQL 保存成字符串 `STAGING_SQL`，再运行：

```python
connection = setup()
try:
    connection.executescript(STAGING_SQL)
    try:
        import_batch(connection, 'R-BATCH-BAD')
    except ValueError as error:
        print(error)
    else:
        raise AssertionError('脏批次没有被拒绝')
    assert connection.execute('SELECT COUNT(*) FROM
      appointments').fetchone()[0] == 0
    assert connection.execute('SELECT COUNT(*) FROM
      appointment_events').fetchone()[0] == 0
    assert connection.execute("SELECT COUNT(*) FROM slots WHERE
      status='available'").fetchone()[0] == 3
    connection.execute("UPDATE appointment_staging SET duration_text='30'
      WHERE line_number=2")
    import_batch(connection, 'R-BATCH-OK')
    print(connection.execute('SELECT id,slot_id FROM appointments ORDER BY
      id').fetchall())
    assert connection.execute('SELECT COUNT(*) FROM
      appointment_events').fetchone()[0] == 3
    assert connection.execute("SELECT COUNT(*) FROM slots WHERE
      status='booked'").fetchone()[0] == 3
finally:
    connection.close()
```

成功时最后打印 `[('A-B1', 'SL1'), ('A-B2', 'SL2'), ('A-B3', 'SL3')]`。错误输出只含行号和类别，不必复制客户信息。TEMP 表属于当前连接，关闭即消失；真实导入若要保留待修正原文，还要明确保存期限与访问范围。

如果业务允许逐行成功，结果就需要分别报告成功与失败行，那是另一种接收规则。这里既然要求整批完整，就不能在循环里每成功一行便提交一次。

取消预约为什么不直接 DELETE？

确认过的 A1 是历史事实。客户改变计划，只说明预约由 confirmed 变为 cancelled，不说明它从未发生。用状态保留原行、在日常列表里过滤掉，是 soft deletion 的一种做法。

取消也有两处相关改变：预约状态与取消事件。使用下面函数，再从新库确认 A1 后调用它：

```python
def cancel(connection, appointment_id, request_id):
    connection.execute('BEGIN')
    try:
        changed = connection.execute("""
            UPDATE appointments SET status='cancelled'
            WHERE id=? AND status='confirmed'
        """, (appointment_id,))
        if changed.rowcount != 1:
            raise ValueError('confirmed_appointment_not_found')
        connection.execute("""
            INSERT INTO appointment_events
              (appointment_id,request_id,event_type)
            VALUES (?,?,'appointment_cancelled')
        """, (appointment_id, request_id))
        connection.execute('COMMIT')
    except Exception:
        if connection.in_transaction:
            connection.execute('ROLLBACK')
        raise
```

调用 `cancel(connection, 'A1', 'R-CANCEL')` 后，A1 仍在，状态是 cancelled；事件表有确认与取消两条事件。日常看板若只想展示有效预约，应使用明确的状态条件；数据库不会因为取消就让所有查询自动隐藏这一行。

本例取消后保留 SL1 的 booked 状态，表示它尚未重新开放。原因是 `appointments.slot_id UNIQUE` 仍把历史 A1 计入唯一性。即使把 SL1 改成 available，第二条预约也会被 UNIQUE 拒绝，还会造成可用展示与实际约束矛盾。真正要支持取消后复用，需要一起调整历史保留、可用状态和唯一规则；当前小模型只承诺取消留痕，不承诺重新放号。

如果想删除客户 C1，先查看真正的引用链：

```sql
SELECT c.id AS customer_id, t.id AS ticket_id,
       a.id AS appointment_id, a.status
FROM customers AS c
LEFT JOIN tickets AS t ON t.customer_id=c.id
LEFT JOIN appointments AS a ON a.ticket_id=t.id
WHERE c.id='C1';
```

能看到 `C1 / T-42 / A1 / cancelled`。直接引用 C1 的是 tickets；预约经工单间接关联客户。因此 `DELETE FROM customers WHERE id='C1';` 首先被工单到客户的 RESTRICT 外键拒绝。取消预约不解除这条历史关系。

`DELETE` 用于确实应移除的行；`RESTRICT` 拒绝留下孤立引用；`CASCADE` 则删除父行时连同关联子行继续删除。下面的临时小表可以直接观察级联范围，它们不属于预约模型：

```SQL
CREATE TEMP TABLE import_batches (id TEXT PRIMARY KEY NOT NULL);
CREATE TEMP TABLE import_lines (
    id INTEGER PRIMARY KEY,
    batch_id TEXT NOT NULL REFERENCES import_batches(id) ON DELETE CASCADE
);
INSERT INTO import_batches VALUES ('B1');
INSERT INTO import_lines VALUES (1,'B1'),(2,'B1');
DELETE FROM import_batches WHERE id='B1';
SELECT COUNT(*) FROM import_lines;
```

结果为 0：这两条临时导入行只从属于该批次，删除范围已经明确。但维修工单和预约需要保留历史，不能为了让 DELETE 通过就改成 CASCADE。SQLite 外键删除行为

一条规则必须对每个写入入口成立，何时需要 trigger?

假设新增了一条明确规则：已取消的历史预约不能直接恢复为 confirmed，需要另行安排。当前状态 CHECK 只能说明新值属于允许集合；它不知道这一行以前是什么状态。

在单独的实验连接上，可以增加一个范围很窄的触发器（trigger）：

```SQL
CREATE TRIGGER prevent_reconfirm
BEFORE UPDATE OF status ON appointments
FOR EACH ROW
WHEN OLD.status='cancelled' AND NEW.status='confirmed'
BEGIN
    SELECT RAISE(ABORT, 'cancelled_appointment_is_final');
END;
```

`OLD.status` 是更新前的值，`NEW.status` 是想写入的值。对已取消 A1 执行 `UPDATE appointments SET status='confirmed' WHERE id='A1';` 时，触发器自动拒绝，A1 继续保持 cancelled。普通 confirmed → cancelled 不符合 WHEN，仍然允许。

这里自动化的是一条依赖前后状态的规则。确认时的三步业务写入仍显式留在 Python 中，因为调用者清楚地提供 request ID，并能指出哪一步失败。触发器引入了另一个自动执行的位置，应当像普通写入一样验证；`RAISE(ABORT)` 拒绝语句，也不替代外层事务的整体撤回。SQLite CREATE TRIGGER

如果再次遇到“SQL 没报错，但事情只做了一半”，先找业务动作的完成条件：哪些表必须共同变化，哪一步真正提交，失败后有没有明确撤回并读回状态。只有出现两个真实连接互相影响的现象，才继续研究锁与隔离；BEGIN 这个入口本身不能代替并发实测。

写入、删除与触发器对应 CS50 SQL Week 3。事务在官方课程的 Week 5 中展开，本章为完成安全写入提前应用这段关系，并不表示官方 Week 3 已经教授完整事务内容。这里引用的操作文档是 live 文档；错误文字、连接默认值和并发行为应随实际 Python 与 SQLite 版本核对。

C05 | 应用怎样安全读到预约，并留下能追踪的线索

先沿客户编号追到固定查询、参数值、结果和 request_id 日志；视图与扩展写法随后再查。

客服输入客户编号 `C-1`，看板应该显示这位客户尚未取消的两次预约。数据库里有四次预约，应用不能把另外一位客户的预约混进来，也不能把已经取消的预约重新显示出来。

先把这两行怎样查出来追清，再让 Python 接过客户编号、执行查询、交回结果。等这条读取走通，我们再解决两种变化：多处程序怎样共用读取规则，某次读取失败后怎样知道发生了什么。

客户编号怎样变成两行看板数据？

这次使用一个独立的读取实验。`customers` 每行是一位客户，`technicians` 每行是一位技术员，`appointments` 每行是一次预约。

客户表有 `C-1 / 林安`、`C-2 / 周宁`；技术员表有 `T-1 / 陈可`、`T-2 / 吴川`。预约表保存它们的编号：

id	customer_id	technician_id	starts_at	status
A-11	C-1	T-1	2026-08-30T09:00:00	confirmed
A-12	C-2	T-1	2026-08-30T10:30:00	confirmed
A-13	C-1	T-2	2026-08-30T14:00:00	confirmed
A-14	C-1	T-2	2026-08-31T09:00:00	cancelled

这里把客户、技术员编号直接放在预约表中，缩短本次读取的关联路径。C03、C04 的持久化模型仍然是从预约经工单找客户：`appointments.ticket_id → tickets.customer_id → customers.id`；从预约经时段找技术员：`appointments.slot_id → slots.technician_id → technicians.id`。**本章另建实验库，不是给 C04 的库自动迁移。**两套结构可以向看板交出相同列名，但关联语句必须服从各自的表。

先直接查询这个小实验：

```sql
SELECT a.id AS appointment_id,
       c.name AS customer_name,
       t.name AS technician_name,
       a.starts_at, a.status
FROM appointments AS a
JOIN customers AS c ON c.id = a.customer_id
JOIN technicians AS t ON t.id = a.technician_id
WHERE a.customer_id = 'C-1'
  AND a.status <> 'cancelled'
ORDER BY a.starts_at, a.id;
```

a、c、t 是这条查询里的表别名。以 A-11 为例：a.customer_id 是 C-1，客户表主键恰好找到林安；a.technician_id 是 T-1，技术员表主键恰好找到陈可。JOIN 把这三处事实组合成一行读取结果，并没有向任何表写入“组合后的新预约”。

这里每个预约最多匹配一位客户和一位技术员，因为被匹配的是各表唯一的主键。外键又保证引用的对象存在，所以这份合法样本的 JOIN 不会丢掉预约，也不会把一次预约展开成多行。

然后筛选：A-12 的客户不是 C-1；A-14 虽然属于 C-1，却已经取消。留下的结果是：

appointment_id	customer_name	technician_name	starts_at	status
A-11	林安	陈可	2026-08-30 T09:00:00	confirmed
A-13	林安	吴川	2026-08-30 T14:00:00	confirmed

结果一行仍代表一次预约。AS appointment_id 把底表的 id 换成消费者容易理解的列名；姓名来自相关表。排序先看时间，时间相同时再看预约编号，避免并列值让顺序不确定。这里的时间文字统一使用同一种格式、同一时区约定，才能直接按文字比较。

如果只有一个调用者，这条直接查询已经足够。SQL 能运行不是终点；现在还知道为什么恰好是这两行、这些列和这个顺序。

看板和通知程序怎样共用同一种读取规则？

假设通知程序也要读取未取消预约。它和看板各抄一段 JOIN，之后一处漏掉取消过滤，就会对同一预约给出不同答案。此时值得把共同的读取定义保存为 view，中文常叫视图。

下面这段稍后保存为 `c05_view.sql`：

```sql
CREATE VIEW upcoming_appointments AS
SELECT a.id AS appointment_id,
       a.customer_id,
       c.name AS customer_name,
       a.technician_id,
       t.name AS technician_name,
       a.starts_at, a.status
FROM appointments AS a
JOIN customers AS c ON c.id = a.customer_id
JOIN technicians AS t ON t.id = a.technician_id
WHERE a.status <> 'cancelled';
```

这里先让入口保存所有未取消预约，再由每次调用筛选客户。这个名字沿用预约看板的叫法；定义里没有日期条件，所以它实际承诺的是“未取消”，并不自动代表“晚于现在”。需要今天或未来时，仍须明确添加日期条件。

view 保存的是查询定义。它没有把 A-11、A-12、A-13 复制成另一张长期数据表；以后客户表姓名改变，下一次读取就会得到更新后的姓名。取消也只是预约行状态改变，底表记录还在，是 view 的过滤规则让它不再出现在看板。

消费者现在可以写：

```sql
SELECT appointment_id, customer_name, technician_name, starts_at, status
FROM upcoming_appointments
WHERE customer_id = 'C-1'
ORDER BY starts_at, appointment_id;
```

它仍取得刚才的两行。看板依赖公开列名和过滤含义；JOIN 的具体拼接由 view 的定义承担。普通 view 也不自动规定读取顺序，所以消费者的 `ORDER BY` 仍在。

这类入口的价值来自稳定复用，不来自 SQL 长度。只有一次的临时查询可以直接写，没必要先给数据库增加一个永久名字。

怎样把这份小数据库准备好，让 Python 真正读一次？

在自己选定的全新实验目录工作。以下程序只使用合成数据，要求 Python 3.12 或更新版本。把下面结构与样本保存为 `c05_schema.sql`，把上一节完整的 `CREATE VIEW` 保存为同目录的 `c05_view.sql`：

SQL

```
CREATE TABLE customers (
    id TEXT PRIMARY KEY NOT NULL,
    name TEXT NOT NULL
);
CREATE TABLE technicians (
    id TEXT PRIMARY KEY NOT NULL,
    name TEXT NOT NULL
);
CREATE TABLE appointments (
    id TEXT PRIMARY KEY NOT NULL,
    customer_id TEXT NOT NULL REFERENCES customers(id) ON DELETE RESTRICT,
    technician_id TEXT NOT NULL REFERENCES technicians(id) ON DELETE RESTRICT,
    starts_at TEXT NOT NULL,
    status TEXT NOT NULL CHECK (status IN ('confirmed', 'cancelled'))
);
INSERT INTO customers VALUES ('C-1', '林安'), ('C-2', '周宁');
INSERT INTO technicians VALUES ('T-1', '陈可'), ('T-2', '吴川');
INSERT INTO appointments VALUES
    ('A-11', 'C-1', 'T-1', '2026-08-30T09:00:00', 'confirmed'),
    ('A-12', 'C-2', 'T-1', '2026-08-30T10:30:00', 'confirmed'),
    ('A-13', 'C-1', 'T-2', '2026-08-30T14:00:00', 'confirmed'),
    ('A-14', 'C-1', 'T-2', '2026-08-31T09:00:00', 'cancelled');
```

把下面完整程序保存为 `c05_setup.py`。它从两个文件读取 SQL，首次建立数据库；目标文件已经存在就停止，不清空旧库。

```python
import os
import sqlite3
from pathlib import Path

root = Path(__file__).resolve().parent
database_path = Path(
    os.environ.get("C05_DATABASE_PATH", str(root / "c05.sqlite3"))
).resolve()
schema = (root / "c05_schema.sql").read_text(encoding="utf-8")
view = (root / "c05_view.sql").read_text(encoding="utf-8")
database_path.touch(exist_ok=False)
connection = sqlite3.connect(database_path, autocommit=True)
try:
    connection.execute("PRAGMA foreign_keys = ON")
    connection.executescript("BEGIN;\n" + schema + "\n" + view + "\nCOMMIT;")
except Exception:
    if connection.in_transaction:
        connection.execute("ROLLBACK")
    raise
finally:
    connection.close()
print("实验库已建立")
```

`root` 指脚本所在目录，SQL 文件据此定位；默认数据库也在该目录。环境变量 `C05_DATABASE_PATH` 可以换成一个新实验文件的绝对路径，其父目录须已存在。环境配置选择“访问哪份数据”，不改变查询的业务含义。若自己设置过该变量，先核对它确实指向新练习文件。

这里的 `executescript` 执行文件中的固定建库脚本，`BEGIN` 与 `COMMIT` 明确包住建表、样本和 view。异常路径显式 `ROLLBACK`，最后总会关闭连接。`autocommit=True` 使这个初始化程序由 SQL 中的事务命令控制边界；它不是下面应用连接的事务设置。不要把客户输入拼进这类脚本。

前面用 `sqlite3` 已经能完成参数化查询；这里换用 SQLAlchemy Core 2.0，是为应用提供集中管理连接、执行查询和按列名取得结果的入口。SQLite 仍负责保存和查询同一份数据，SQL 仍表达读取规则；增加的这一层负责 Python 怎样调用它们，简单脚本继续直接用 `sqlite3` 也成立。

在该实验目录的 PowerShell 中建立局部依赖环境并运行初始化：

```powershell
python -m venv .venv
if ($LASTEXITCODE -ne 0) { throw '虚拟环境建立失败' }
& .\.venv\Scripts\python.exe -m pip install 'SQLAlchemy==2.0.52'
if ($LASTEXITCODE -ne 0) { throw '依赖安装失败' }
& .\.venv\Scripts\python.exe .\c05_setup.py
if ($LASTEXITCODE -ne 0) { throw '初始化失败，请核对报错与实验文件' }
```

这是一次性初始化；后面反复读库时不必重新运行它。若初始化中断，目标文件可能已经保留，请选择另一个新实验文件再试，不把未知文件当作可清空对象。

先保存下面完整程序为 `c05_read.py`：

```python
import os
from pathlib import Path

from sqlalchemy import URL, create_engine, event, text


def make_engine(database_path):
    database_path = Path(database_path).resolve()
    if not database_path.is_file():
        raise FileNotFoundError("实验数据库不存在，请先初始化")
    engine = create_engine(
        URL.create("sqlite+pysqlite", database=str(database_path)),
        connect_args={"autocommit": False},
        hide_parameters=True,
    )

    @event.listens_for(engine, "connect")
    def enable_foreign_keys(dbapi_connection, connection_record):
        previous = dbapi_connection.autocommit
        dbapi_connection.autocommit = True
        try:
            cursor = dbapi_connection.cursor()
            try:
                cursor.execute("PRAGMA foreign_keys = ON")
                if cursor.execute("PRAGMA foreign_keys").fetchone()[0] != 1:
                    raise RuntimeError("无法启用外键检查")
            finally:
                cursor.close()
        finally:
            dbapi_connection.autocommit = previous

    return engine
```

PYTHON · 续

```
CUSTOMER_QUERY = text("""
    SELECT appointment_id, customer_name, technician_name, starts_at, status
    FROM upcoming_appointments
    WHERE customer_id = :customer_id
    ORDER BY starts_at, appointment_id
""")


def list_customer_appointments(engine, customer_id):
    with engine.connect() as connection:
        result = connection.execute(CUSTOMER_QUERY, {"customer_id":
          customer_id})
        rows = [dict(row) for row in result.mappings().all()]
    return rows


if __name__ == "__main__":
    root = Path(__file__).resolve().parent
    path = os.environ.get("C05_DATABASE_PATH", str(root / "c05.sqlite3"))
    engine = make_engine(path)
    try:
        rows = list_customer_appointments(engine, "C-1")
        print([row["appointment_id"] for row in rows])
    finally:
        engine.dispose()
```

运行它：

POWERSHELL

```
& .\.venv\Scripts\python.exe .\c05_read.py
```

终端应该显示 `['A-11', 'A-13']`。下一次只需要重新运行读取程序，已经保存的表与 view 仍在数据库文件中。

这段程序里，谁接过参数，谁交回结果？

先抓住执行查询的三行。函数接到 `customer_id="C-1"`，把值放进字典交给 `execute`；数据库返回结果后，函数再把它变成 Python 字典列表，`return rows` 交给调用者。函数的参数是这次工作的输入，返回值是调用者下一步能够使用的结果，二者不是同一个东西。

`CUSTOMER_QUERY` 是固定的 SQL 语句对象。`text()` 把 SQL 文字包装为 SQLAlchemy 可执行的对象，并识别 `:customer_id` 这个参数位置；它不会自动读库。真正发起执行的是 `connection.execute(...)`。

`execute` 返回 `Result`，还不是最终的页面数据。`mappings()` 让结果的每一行按列名取值，例如 `row["appointment_id"]`；`all()` 取完当前结果；`dict(row)` 把每行复制成普通字典。列表中的第一项因此含有 `appointment_id="A-11"`、`customer_name="林安"` 等五个字段。只打印编号是为了在终端容易核对，函数交回的仍是完整五列。这种全部取回适合本章小结果，巨大结果需要另定分批或分页方案。

点号后的 `connect()`、`execute()`、`mappings()` 都是对象的方法：分别让 engine、connection、result 做各自负责的动作。无需先建立 ORM 类，也不需要把数据库的一行想成会自动同步回数据库的 Python 对象；改返回字典并不会改表。

`create_engine` 建立连接的管理入口，`URL.create` 告诉它使用 SQLite 驱动并访问配置中的文件。创建 engine 通常还没有真正打开文件；进入 `with engine.connect()` 才取得这次使用的连接。离开 `with` 就释放本次使用权，并结束尚未提交的事务；底层连接可能归回连接池复用。`engine.dispose()` 用在整个实验收尾，释放 engine 当前持有的空闲连接。这里不需要调整连接池容量，先保证一次调用用完就归还。

为什么还有 `enable_foreign_keys` 那段？SQLite 的外键开关属于每条底层连接。初始化程序开过一次，并不等于应用以后新开的连接也开启。装饰器 `@event.listens_for(engine, "connect")` 把函数登记为“每次创建底层连接时运行”；它收到驱动连接，在开始使用前打开并读回外键开关。`connection_record` 是 SQLAlchemy 传来的管理信息，这个实验不用它。

应用的 `autocommit=False` 采用 Python 3.12 起提供的事务控制，使读取和后面的 DDL 迁移都有明确事务。SQLite 不能在已经开始的事务中有效切换外键开关，所以回调暂时设置 `autocommit=True`，执行 `PRAGMA` 后恢复原设置。游标只负责执行这几条底层命令，用完关闭。这里需要记住的是“每条新连接都要配置，且在事务外设置”，具体回调写法可以据官方文档查回；不要把一段设置代码当作所有数据库都相同的规律。

这份实验现在有两条能核对的交接：文件配置决定打开哪个库，客户参数决定在该库中选哪些行。路径写错和客户编号写错属于不同问题，不能都用“查不到数据”解释。

客户编号里带引号，为什么也只能是一个值？

把函数调用中的 `"C-1"` 换成 `"C-1' OR 1=1 --"`，查询应该返回空列表。原因不是数据库删除了可疑字符，而是整个字符串始终处在一个参数位置中：数据库寻找客户编号恰好等于这整段文字的行，样本里没有。

反过来看一个错误的拼接结果。若程序用字符串格式化把客户输入塞在 SQL 引号之间，就可能得到这条语句：

```sql
SELECT appointment_id
FROM upcoming_appointments
WHERE customer_id = 'C-1' OR 1=1 --'
ORDER BY starts_at, appointment_id;
```

输入中的引号结束了原字符串；`OR 1=1` 变成对每行都成立的条件；`--` 把后面的残余引号变成注释。于是 A-12 也会被选中。这是演示错误边界的语句，只在上述合成数据里观察它；应用函数始终使用绑定参数。

绑定保护 SQL 结构，授权决定这次调用能看谁的数据。即使使用绑定，传入合法的 C-2 仍会读到周宁的预约。若登录者只能看自己的预约，应用必须先从可信登录身份确定允许的客户编号，或在查库前核对访问权，不能让任意请求里的 `customer_id` 自己证明权限。本章函数负责已授权后的读取，不包含登录系统。

参数位置承载的是值，不是列名、表名或 `ASC/DESC` 语法。如果页面允许选择排序方式，应用应把有限选项映射到自己写好的查询结构；不能把用户文字直接拼成 SQL，再指望参数绑定替它补救。

view 同样不授予访问权。若进程能直接读取底表，它可以绕过 view 的过滤。SQLite 实验主要由文件访问权限和应用规则约束；服务器数据库还需配置运行账号的最小权限，并把改表能力与普通请求分开。数据库账号、真实服务端权限和连接池调优，要在目标环境出现时再学习，本章不据本地文件实验推断它们已经成立。

查询失败了，怎样把线索关联到那一次处理?

现在函数能交回数据，但运行中只看到“查询失败”还不够。客服连续打开两次相同客户的看板，业务编号仍是 C-1，却已经是两次处理。要把某一次的入口、数据库操作和最终回应对应起来，需要另一个关联标识 `request_id`。

它代表一次 HTTP 请求或一次任务处理，由可信的应用入口创建，再向下传递。同一次处理做多项工作时共享它；下一次处理即使读同一客户，也使用新的标识。**request ID 关联一次处理，不等于客户或预约的业务 ID，也不负责防止重复业务效果。**

下面给现有查询加一层观察，不改变其 SQL。保存为 `c05_observe.py`，与 `c05_read.py` 放在同一目录：

PYTHON

```
import json
import os
from pathlib import Path
from time import perf_counter
from uuid import UUID, uuid4

from sqlalchemy.exc import SQLAlchemyError
from c05_read import make_engine, list_customer_appointments


def read_for_request(engine, customer_id, request_id):
    if not isinstance(request_id, UUID):
        raise TypeError("request_id 须由应用入口生成")
    started = perf_counter()
```

PYTHON · 续

```
    try:
        rows = list_customer_appointments(engine, customer_id)
    except SQLAlchemyError as error:
        entry = {
            "request_id": str(request_id),
            "operation": "appointments.list_by_customer",
            "outcome": "error",
            "row_count": None,
            "elapsed_ms": round((perf_counter() - started) * 1000, 3),
            "error_type": type(error).__name__,
        }
        print(json.dumps(entry, ensure_ascii=False))
        raise RuntimeError("预约暂时不可用") from None
    entry = {
        "request_id": str(request_id),
        "operation": "appointments.list_by_customer",
        "outcome": "success",
        "row_count": len(rows),
        "elapsed_ms": round((perf_counter() - started) * 1000, 3),
        "error_type": None,
    }
    print(json.dumps(entry, ensure_ascii=False))
    return rows


if __name__ == "__main__":
    root = Path(__file__).resolve().parent
    path = os.environ.get("C05_DATABASE_PATH", str(root / "c05.sqlite3"))
    engine = make_engine(path)
    request_id = uuid4()
    try:
        rows = read_for_request(engine, "C-1", request_id)
        print([row["appointment_id"] for row in rows])
    except RuntimeError as error:
        print(str(error))
    finally:
        engine.dispose()
```

运行 `& .\.venv\Scripts\python.exe .\c05_observe.py`，会先看到一条 JSON 日志，再看到两个预约编号。`uuid4()` 在应用入口生成新的随机标识；它不是从姓名、电话或请求正文截取出来的。这里用 `UUID` 类型约束入口，避免把任意字符串直接当日志关联字段；类型检查本身不验证用户身份，真实接口仍须管理标识来源。

日志中的 `operation` 是固定操作名，便于将同类读取放在一起看；`outcome` 区分正常结果和失败；`elapsed_ms` 是从调用查询函数到取完行、结束连接使用的总耗时，不是数据库内部纯 SQL 时间；`error_type` 成功时为空，失败时只保留异常类别。`perf_counter()` 是测量经过时间的计时器，两次读数之差乘 1000 得到毫秒，不是预约日期。

成功返回空列表时，`row_count=0` 是确知的结果。数据库失败时还不知道匹配多少行，所以日志用 `row_count=null`。如果把两者都写成零，就会把“查过但没有”与“未能查出答案”混在一起。

这段代码逐个选取安全字段。客户编号、姓名、返回正文、SQL、参数字典、连接配置都没有复制进日志；失败时也没有 `str(error)` 或 `logger.exception(...)` 去展开原始数据库异常。SQLAlchemy 异常可能携带 SQL、底层报错和参数，`hide_parameters=True` 只能提供一部分保护，不能代替这里的字段选择。`raise ... from None` 向调用者交出固定错误提示，常规 traceback 不再串出原数据库异常；对这类错误，外层也不应另行收集异常内部上下文或局部变量中的数据。

一次成功的关系已经闭合：应用入口产生一个 request ID，把客户编号交给读取函数；固定 SQL 绑定编号，view 产出两条预约；函数取完并释放连接；日志保存同一次标识与成功行数；调用者取得列表继续组织页面。日志负责描述这次运行，不是保存业务事实的另一张预约表，也不与数据库事务自动同成同败。

怎样看到真实失败，又不把异常内容写进日志？

仅在本章合成库中，可以临时移除 view 来制造确定的查询失败。下面程序保存为 `c05_failure.py`；它依赖本章刚给出的三个文件，结束时会恢复 view：

PYTHON

```
import os
from pathlib import Path
from uuid import uuid4

from sqlalchemy import text
from c05_read import make_engine
from c05_observe import read_for_request

root = Path(__file__).resolve().parent
path = os.environ.get("C05_DATABASE_PATH", str(root / "c05.sqlite3"))
view_sql = (root / "c05_view.sql").read_text(encoding="utf-8")
engine = make_engine(path)
request_id = uuid4()
try:
    with engine.begin() as connection:
        connection.execute(text("DROP VIEW upcoming_appointments"))
    try:
        read_for_request(engine, "C-1' OR 1=1 --", request_id)
    except RuntimeError as error:
        print(str(error))
    finally:
        with engine.begin() as connection:
            connection.execute(text(view_sql))
finally:
    engine.dispose()
```

运行 `& .\.venv\Scripts\python.exe .\c05_failure.py`，应得到一条 `outcome="error"`、`error_type="OperationalError"` 的 JSON，以及“预约暂时不可用”。日志里的标识与这次调用使用的是同一个，`row_count` 为 `null`；输入中的整段文字不会出现在日志或固定提示里。程序在 `finally` 中恢复读取定义，可以再运行普通读取程序核对两行恢复。

这次失败不是客户不存在：如果只是没有匹配，函数应成功返回 `[]`。这里最早出错的是执行查询时找不到 view，结果行尚未形成。request ID 帮我们把失败关联到这一轮，操作名帮我们定位到预约读取；接下来应检查入口定义与数据库结构。改客户编号不能补回缺失的 view。

`with engine.begin()` 与前面的 `connect()` 相比，还替这段工作管理提交：块正常结束就提交，异常离开就回滚。这个实验先提交移除，才让另一次读取确实看见失败；之后另开事务恢复。它只是受控失败演示，不应在真实业务库中照做。以下迁移实验会进一步把变更和核对放进同一个事务，避免留下这种半成品。

每位技术员的预约数，与每条预约的组内编号，是同一个结果吗？

现在看板读取与运行线索已经清楚。主管另外提出两个报告需求，它们看似都“按技术员分组”，结果一行的含义却不同。

第一个需求是长期共用的工作量汇总。可以把聚合查询也保存为 view：

```sql
CREATE VIEW technician_load AS
SELECT technician_id, COUNT(*) AS open_count
FROM upcoming_appointments
GROUP BY technician_id;
```

```sql
SELECT technician_id, open_count
FROM technician_load
ORDER BY technician_id;
```

结果只有两行：T-1 的 `open_count=2`，T-2 的 `open_count=1`。这里 `open_count` 数的是本实验“未取消”入口中的预约；它不是在筛选一个叫 `open` 的状态，也不是所有历史预约总数。这个聚合从已有预约出发，没有未取消预约的技术员不会出现；若要显示他们的零值，就需从技术员表出发，用 `LEFT JOIN` 并计数非空预约主键。

第二个需求只做一次：列出 2026-08-30 的预约明细，并标出每位技术员当天的第几次预约。直接在本次查询中写：

```sql
WITH filtered AS (
    SELECT appointment_id, technician_id, starts_at
    FROM upcoming_appointments
    WHERE starts_at >= '2026-08-30T00:00:00'
      AND starts_at < '2026-08-31T00:00:00'
)
SELECT appointment_id, technician_id, starts_at,
       ROW_NUMBER() OVER (
           PARTITION BY technician_id
           ORDER BY starts_at, appointment_id
       ) AS row_no
FROM filtered
ORDER BY technician_id, row_no;
```

`WITH filtered AS (...)` 是普通 CTE：给当前语句中的一个中间结果起名字。括号内先限定当天时间，用包含起点、不包含次日起点的区间，得到 A-11、A-12、A-13。`filtered` 的名字只在这条语句内有效，下一条独立 SQL 不能再把它当已保存的 view。

接着看 ROW_NUMBER()：它为行编号；OVER (...) 说明编号时怎样看周围的行。PARTITION BY technician_id 把编号计算分成 T-1、T-2 两组，每组从 1 重新开始。窗口里的 ORDER BY 决定组内谁先得到编号；语句最外面的 ORDER BY 决定最终如何展示结果。前者不代替后者。

appointment_id	technician_id	starts_at	row_no
A-11	T-1	2026-08-30T09:00:00	1
A-12	T-1	2026-08-30T10:30:00	2
A-13	T-2	2026-08-30T14:00:00	1

GROUP BY 把多条明细折成一组一行；window 在明细旁增加计算结果，预约行仍保留。这里三条预约还是三条，不是变成两位技术员。把前一需求的聚合结果交给这一需求，会丢掉预约编号和时间；用 DISTINCT 也找不回来。

view 和 CTE 则是另一个选择维度：是否保存一个以后继续使用的名字。view 里可以写普通查询，也可以写聚合；CTE 后面同样可以继续聚合或做窗口计算。它们不是互斥的三种"分组工具"，更不是承诺自动加速的三种方式。

若要在电脑运行上面三段 SQL，可以把它们按顺序保存为同目录的 c05_reports.sql，再保存并运行 c05_reports.py：

```python
import os
from pathlib import Path

from sqlalchemy import text
from c05_read import make_engine

root = Path(__file__).resolve().parent
path = os.environ.get("C05_DATABASE_PATH", str(root / "c05.sqlite3"))
statements = (root / "c05_reports.sql").read_text(encoding="utf-8")
engine = make_engine(path)
try:
    with engine.begin() as connection:
        for sql in statements.split(";"):
            if not sql.strip():
                continue
            result = connection.execute(text(sql))
            if result.returns_rows:
                print([dict(row) for row in result.mappings().all()])
finally:
    engine.dispose()
```

运行 `& .\.venv\Scripts\python.exe .\c05_reports.py`，建立聚合 view、读取聚合 view、CTE 报告这三段 SQL 顺序执行，会打印两行汇总和三行明细。这个小执行器仅适合这几条没有字符串内分号的固定语句，分号拆分不是通用 SQL 解析器。聚合 view 只创建一次，再跑报告时可以去掉文件中的建 view 语句。`returns_rows` 用来区分返回行的查询和本次不返回行的建 view 操作。

底表要增加姓名列，怎样保住消费者已经依赖的入口？

最后考虑维护中的变化。客户表以后需要独立的显示姓名 `display_name`。为了让现有看板继续使用 `customer_name`，可以新增底表列、复制原姓名，然后让 view 用新列提供同一个公开列名。数据库不会替我们理解这层承诺；更改时必须一起处理依赖。

本次采取“新增并回填，保留旧列”的窄迁移，不猜测怎样把中文姓名拆成姓和名，也不重建整张有外键引用的客户表。保存下面程序为 `c05_migrate.py`：

PYTHON

```
import os
from pathlib import Path

from sqlalchemy import text
from c05_read import make_engine

PUBLIC_COLUMNS = [
    "appointment_id", "customer_id", "customer_name",
    "technician_id", "technician_name", "starts_at", "status",
]


def migrate_display_name(engine, fail_after_drop=False):
    root = Path(__file__).resolve().parent
    old_view = (root / "c05_view.sql").read_text(encoding="utf-8")
    new_view = old_view.replace(
        "c.name AS customer_name", "c.display_name AS customer_name"
    )
    if new_view == old_view:
        raise RuntimeError("读取定义与这份迁移不匹配")
    with engine.begin() as connection:
        before_result = connection.execute(text(
            "SELECT * FROM upcoming_appointments ORDER BY appointment_id"
        ))
        before = before_result.all()
        if list(before_result.keys()) != PUBLIC_COLUMNS:
            raise RuntimeError("原读取列名已经偏离约定")
        connection.execute(text(
            "ALTER TABLE customers ADD COLUMN display_name TEXT"
        ))
        connection.execute(text("UPDATE customers SET display_name = name"))
```

PYTHON · 续

```
        connection.execute(text("DROP VIEW upcoming_appointments"))
        if fail_after_drop:
            raise RuntimeError("演示：迁移在重建入口前中断")
        connection.execute(text(new_view))
        after_result = connection.execute(text(
            "SELECT * FROM upcoming_appointments ORDER BY appointment_id"
        ))
        after = after_result.all()
        if list(after_result.keys()) != PUBLIC_COLUMNS or after != before:
            raise RuntimeError("迁移改变了消费者看见的列或数据")


if __name__ == "__main__":
    root = Path(__file__).resolve().parent
    path = os.environ.get("C05_DATABASE_PATH", str(root / "c05.sqlite3"))
    engine = make_engine(path)
    try:
        migrate_display_name(engine)
        print("读取列与样本结果保持一致")
    finally:
        engine.dispose()
```

运行命令是 `& .\.venv\Scripts\python.exe .\c05_migrate.py`。这里的 `replace` 只修改随本章提供的固定 SQL 定义，不接收用户输入。新列名字是 `display_name`，公开表达式仍写 `AS customer_name`；消费者因此不用猜底表后来改成什么名字。

迁移正常结束前，程序比较公开列名及已知结果，姓名、取消过滤、预约数量都在这些样本结果中得到核对。任何一步抛出异常都会退出 `engine.begin()` 并回滚，新增列、回填、移除与重建 view 作为一组改变处理。这里特意使用已说明的非旧式事务控制；只写一个 Python `with`，却让旧版驱动在 DDL 前未开始事务，不能据此声称结构变更可回滚。

若想观察中断，在一份尚未迁移的本章实验库上，把主程序调用改为 `migrate_display_name(engine, fail_after_drop=True)`。它会在 view 被移除后抛错，事务回滚。重新运行普通读取仍应得到 A-11、A-13，客户表也不会留下新增列。再改回默认调用，完整迁移后应输出“读取列与样本结果保持一致”。已经成功迁移的库不要再执行同一新增列步骤；重复迁移管理是另一项工作。

保留旧列意味着以后若增加姓名写入口，还必须明确由哪个列保存新的显示姓名；本章只验证一次读取迁移，并不承诺两列会自行同步。前面的移除 view 失败实验用于迁移前的基线；迁移后不应拿旧 `c05_view.sql` 再覆盖已经更新的定义。

可以用一个局部改动检验自己是否追得上：把新 view 表达式里的 `AS customer_name` 改成 `AS display_name`，迁移应当在哪里停止？它会在事务结束前的公开列名比较处失败，随后恢复旧结构。这时新底表列有没有建立成功并不足以判断整件事成功，因为消费者需要的交接已经改变。

到这里，稳定读取入口、参数绑定、授权和日志各有具体负责的事情。下一章若日志里的同一类读取持续耗时很高，先保留这些正确行与固定顺序，再用查询计划和重复实测寻找原因；耗时下降不能弥补数据读错。

忘记具体接口写法时，从哪里查回？

SQLAlchemy 2.0.52 是本实验固定依赖。Python 3.12+ 的事务参数、每连接外键设置和执行接口以官方说明为准；升级解释器、库或换数据库后，要重新核对参数、回滚、列名与已知结果。以下为持续更新的官方文档入口，技术写法复核于 2026-09-07：

- SQLAlchemy：连接、事务、结果映射与绑定参数，用于查 `connect()`、`begin()`、`text()` 和 `mappings()` 的调用关系。
- SQLAlchemy：SQLite 事务与外键设置，用于查 `autocommit=False` 以及每条底层连接启用外键的回调。
- Python sqlite3：事务控制 与 参数占位符，用于区分脚本事务、驱动事务与绑定值。
- SQLite：CREATE VIEW、WITH、窗口函数，用于查入口定义、名字寿命和明细计算。
- SQLite：外键，用于查连接开关及事务内设置的限制。

C06 | 结果对了以后，怎样判断慢在哪里

先确认结果正确，再用同一条件建立基线、看计划、做实测；速度不能替正确性作证。

客户打开历史预约，页面等了一会儿才显示列表。上一章的读取事件可以把这次等待缩小到一个操作：

JSON
```
{
  "request_id": "7b8512d3-16f2-4e08-8cb0-22e57788c340",
  "operation": "appointments.history_by_customer",
  "outcome": "success",
  "row_count": 20,
  "elapsed_ms": 186.4,
  "error_type": null
}
```

这是一个假设现场，沿用上一章的事件字段和 UUID 形式的请求标识，操作换成了读取客户历史：一次读取交回 20 行，用时 186.4 毫秒，`outcome` 为成功，`error_type` 为空。这个数字还不能告诉我们答案对不对、是否每次都慢，以及时间花在了哪里。本章从固定正确答案开始，再观察数据库怎样找到这些行，最后用相同条件下的多次运行判断一个索引是否值得保留。

先看这条事件的计时范围。如果代码把获取连接、查询和整理结果一起计时，186.4 毫秒就是这一整段的时间；不能全部记在 SQL 身上。`request_id` 帮我们找到同一次处理的证据，`operation` 帮我们比较同类操作。反复出现同类慢事件后，才把对应 SQL、输入和有代表性的数据拿出来复现；一个偶然的慢样本不足以定位原因。

“最近 20 条”究竟应该是哪 20 条？

这一章单独建一个只有四列的预约表，用来观察查找路线：

SQL
```
CREATE TABLE appointments (
    id INTEGER PRIMARY KEY,
    customer_id TEXT NOT NULL,
    starts_at TEXT NOT NULL,
    status TEXT NOT NULL CHECK (status IN ('confirmed', 'completed'))
);
```

这里的 `customer_id` 直接放在预约上，只用于本章独立的性能实验。C03/C04 的真实关系仍然是预约经工单找到客户、经时段找到技术员；本章没有迁移或替换那套表。

先给客户 C-TARGET 放入下面 30 条预约。编号的末两位从 01 到 30；前 28 条分别在八月 1 日到 28 日，最后两条都在八月 30 日。前 24 条状态为 completed，其余为 confirmed。

id	customer_id	starts_at
100001	C-TARGET	2026-08-01
100002	C-TARGET	2026-08-02
…	…	…
100028	C-TARGET	2026-08-28
100029	C-TARGET	2026-08-30
100030	C-TARGET	2026-08-30

需求是“这位客户最近的 20 次预约”，不按状态排除其中任何一条。同一天的预约约定按编号从大到小显示。查询因此是：

```sql
SELECT id, customer_id, starts_at, status
FROM appointments
WHERE customer_id = ?
ORDER BY starts_at DESC, id DESC
LIMIT 20;
```

? 绑定 C-TARGET，先限定客户。starts_at DESC 表达从新到旧，id DESC 决定同一天的两条谁在前，最后取前 20 条。编号是唯一主键，因此第二个排序键消除了并列的不确定性。本例日期统一使用 YYYY-MM-DD，文字顺序与日期顺序相同。

不用运行就能推出结果 ID 应为 100030，100029，…，100011。其中开头两条日期相同，100030 必须在 100029 前。只核对“有 20 行”会漏掉客户、边界和顺序错误；只写 ORDER BY starts_at DESC 则没有规定同日两行的相对位置。建索引可能改变访问路线，不能让偶然的读出顺序替我们定义答案。

这就是后面比较的起点：无论数据库采用什么办法，返回的四列、这 20 个 ID 及其顺序都必须保持。

表里另外有十万条预约时，它怎样找到这位客户？

现在保持这 30 条不变，再加入 100,000 条其他客户的预约。业务答案没有变化，但没有客户查找结构时，数据库可能需要检查整张表，逐行判断 customer_id 是否相等。

可以在同一查询前面加上 `EXPLAIN QUERY PLAN`，请 SQLite 描述访问计划：

```sql
EXPLAIN QUERY PLAN
SELECT id, customer_id, starts_at, status
FROM appointments
WHERE customer_id = 'C-TARGET'
ORDER BY starts_at DESC, id DESC
LIMIT 20;
```

这里使用固定合成文字是为了便于直接运行；应用里的输入仍使用参数绑定。计划里可能看到：

```
SCAN appointments
USE TEMP B-TREE FOR ORDER BY
```

第一行说明扫描预约表。第二行说明这个计划还需要临时 B-tree 结构来处理排序；此刻把它理解为“数据库另做了一份排序工作”即可。

`SCAN` 不是报错，也不等于“完全没有使用索引”。SQLite 用这个词描述遍历全部记录的路线，其中也可能包含沿某个索引顺序遍历。相对地，`SEARCH` 表示按条件定位，只访问表中一部分行。看计划时要连同表名、索引名和使用的条件一起读，不能只凭一个英文词评判好坏。完整输出格式会随 SQLite 版本改变。SQLite 的计划说明解释了这两种访问方式与临时排序的含义。

当前条件只需要 100,030 行中的 30 行。我们因此提出一个具体候选：按 `customer_id` 建立查找结构，让数据库有机会先定位这一小组预约。

```sql
CREATE INDEX idx_appointments_customer_id
ON appointments(customer_id);
```

`idx_appointments_customer_id` 是这份索引的名字，`ON appointments(customer_id)` 指定它跟随哪张表、哪一列维护。它不创建第二份业务预约，也不要求应用改用另一条查询。原来的 `SELECT` 仍然表达相同需求，数据库选择怎样借助索引完成它。

建好后再次解释同一查询，计划可能包含：

```
SEARCH appointments USING INDEX idx_appointments_customer_id (customer_id=?)
USE TEMP B-TREE FOR ORDER BY
```

现在能看见客户等值条件参与了索引查找。但索引只按客户组织条目，没有承诺完成 `starts_at DESC, id DESC` 的排序。因此第二行仍在，并不表示这次改变失败。它可能已经把原先需要检查的大量无关预约缩小到目标客户的 30 条，再处理这些行的排序。

到这里有了减少无关行访问的理由。至于实际能省多少时间，还需要运行同一条查询来回答。SQLite 查询规划说明用查找与排序的例子解释了索引为什么针对具体查询起作用。

计划变了，怎样知道这次改变有没有用?

计划描述数据库准备怎样访问数据；计时记录这条操作在当前条件下实际用了多久。它们承担不同工作：

要判断的事	要看的证据
仍在回答同一个业务问题吗	已知结果及顺序；建索引前后逐行相同
查找路线发生什么改变	同一 SQL 和参数的前后计划
当前环境是否获得收益	相同计时范围、数据和重复方式下的前后耗时

第一次运行可能包含初始化或缓存尚未准备好的成本；后台进程也会抢占时间。所以两边都先预热一次，再重复执行同样次数，保留每次耗时。取中位数 median 能减少极端样本对代表值的影响，但不能消除所有噪声，更不能代表最慢请求。若系统关心偶发长等待，还需进一步观察慢样本分布。

下面的完整程序只需 Python 标准库。保存为自选实验目录中的 `c06_history.py`，执行 `python .\c06_history.py`。数据库只在内存里存在，结束时关闭。

程序先验证结果，再记录前后计划和 25 次样本。建库、生成数据和创建索引都不计入读取耗时；读取计时包含执行查询并用 `fetchall()` 取完结果，不包含后面的断言和 JSON 输出。这个范围和应用事件未必一样，因此不能把两个数直接相减来归因。

PYTHON

```
import json
import platform
import sqlite3
from statistics import median
from time import perf_counter_ns


QUERY = """
SELECT id, customer_id, starts_at, status
FROM appointments
WHERE customer_id = ?
ORDER BY starts_at DESC, id DESC
LIMIT 20
"""
PARAMETERS = ("C-TARGET",)
EXPECTED_IDS = list(range(100_030, 100_010, -1))
```

PYTHON · 续

```python
def setup(other_count=100_000):
    connection = sqlite3.connect(":memory:")
    connection.execute("""
        CREATE TABLE appointments (
            id INTEGER PRIMARY KEY,
            customer_id TEXT NOT NULL,
            starts_at TEXT NOT NULL,
            status TEXT NOT NULL CHECK (status IN ('confirmed', 'completed'))
        )
    """)
    # 其他客户使用负数 ID；改变数量不会与目标客户 ID 冲突。
    other_rows = [
        (-number, f"C-{number % 1000:04d}", "2025-08-01",
         "confirmed" if number % 2 == 0 else "completed")
        for number in range(1, other_count + 1)
    ]
    target_rows = [
        (100_000 + day, "C-TARGET",
         "2026-08-30" if day in (29, 30) else f"2026-08-{day:02d}",
         "completed" if day < 25 else "confirmed")
        for day in range(1, 31)
    ]
    connection.executemany(
        "INSERT INTO appointments VALUES (?, ?, ?, ?)",
        other_rows + target_rows,
    )
    connection.commit()
    return connection


def fetch_history(connection):
    return connection.execute(QUERY, PARAMETERS).fetchall()


def check_result(rows):
    assert [row[0] for row in rows] == EXPECTED_IDS
    assert all(row[1] == "C-TARGET" for row in rows)
    assert rows[0][2] == rows[1][2] == "2026-08-30"


def explain(connection):
    rows = connection.execute("EXPLAIN QUERY PLAN " + QUERY, PARAMETERS)
    return [row[3] for row in rows]
```

PYTHON · 续

```
def measure(connection, expected_rows, repeats=25):
    assert fetch_history(connection) == expected_rows  # 同样预热一次
    samples = []
    for _ in range(repeats):
        started = perf_counter_ns()
        rows = fetch_history(connection)
        elapsed_ms = (perf_counter_ns() - started) / 1_000_000
        assert rows == expected_rows
        samples.append(elapsed_ms)
    return {"samples_ms": samples, "median_ms": median(samples)}


def main(other_count=100_000):
    connection = setup(other_count)
    try:
        before_rows = fetch_history(connection)
        check_result(before_rows)
        before_plan = explain(connection)
        before_time = measure(connection, before_rows)

        connection.execute(
            "CREATE INDEX idx_appointments_customer_id "
            "ON appointments(customer_id)"
        )
        connection.commit()
        after_rows = fetch_history(connection)
        check_result(after_rows)
        assert after_rows == before_rows
        after_plan = explain(connection)
        after_time = measure(connection, before_rows)

        print(json.dumps({
            "python": platform.python_version(),
            "sqlite": sqlite3.sqlite_version,
            "operation": "appointments.history_by_customer",
            "total_rows": other_count + 30,
            "sql": QUERY.strip(),
            "parameters": PARAMETERS,
            "result_ids": [row[0] for row in after_rows],
            "result_unchanged": after_rows == before_rows,
            "before_plan": before_plan,
            "after_plan": after_plan,
            "warmups_per_phase": 1,
            "repeats_per_phase": 25,
            "before": before_time,
            "after": after_time,
        }, ensure_ascii=False, indent=2))
```

PYTHON · 续

```
    finally:
        connection.close()


if __name__ == "__main__":
    main()
```

这里省略连接的事务参数，使用 `sqlite3` 当前默认的旧事务控制模式（`LEGACY_TRANSACTION_CONTROL`）：初始化的 `INSERT` 会隐式开启事务，随后 `connection.commit()` 提交这些数据，计时才开始观察读取。创建索引本身没有隐式开启一段待提交事务，因此后面的 `commit()` 在此处没有额外工作。C04 明确设置了 `autocommit=True`，在那里应由 SQL `BEGIN/COMMIT/ROLLBACK` 管理业务动作，不能把这两种连接模式与提交写法交叉拼用。本章独立实验只准备数据并比较读取，不验证多步写入或迁移的原子性；升级 Python 时，应核对默认模式是否变化。

这份输出是合成实验的复现记录，所以包含固定 SQL、合成参数和结果 ID。它与上一章应用日志的用途不同；不能把真实客户参数原样加入常规日志。`perf_counter_ns()` 用于测量两次读数之间的时间间隔，除以一百万换成毫秒；`fetchall()` 确保没有只计到开始返回结果的位置。完整 API 写法可在使用时查 Python sqlite3、time和 statistics。

看到输出时，先核对 `result_ids` 和 `result_unchanged`，再解释两份计划，最后比较两组样本及中位数。运行成功不会固定一个“必须快几倍”的答案：SQLite 版本、电脑负载与缓存都会影响数值。一次内存实验也没有测量磁盘读写、连接池等待或整次 HTTP 请求。

为了检验自己是否理解了这条关系，把程序最后的 `main()` 改成 `main(other_count=100)`。运行前先说出：哪份结果必须相同，哪种工作减少了，哪个数字还不能预先断言？

目标客户的 30 条预约没动，期望 ID 仍是 `100030` 到 `100011`；无索引时要检查的其他客户行从十万降到一百。小表的扫描成本随之缩小，索引的绝对收益可能已经很小。是否更快、计划是否仍选择相同路线，由新输出回答。能解释这个预测，再用程序验证，比只认得 `CREATE INDEX` 更接近实际会用；这里不需要默写数据生成代码。

如果结果数量本来就错了，应该先改哪里？

假设另一份查询想列出每条预约所属的客户。已知两位客户 `C-1`、`C-2`，三条预约分别是 `A-1→C-1`、`A-2→C-1`、`A-3→C-2`。

SQL

```
SELECT c.id, a.id
FROM customers AS c
JOIN appointments AS a ON c.id = c.id;
```

这个错误条件只比较客户 ID 与自己，没有约束预约属于谁。于是 C-1 配上全部三条预约，C-2 也配上全部三条，共六行，其中包括错误组合 (C-1, A-3)。

哪怕索引让它立即返回，这六行仍然错。六个客户与预约的组合彼此不同，给两列加 DISTINCT 也不会去掉它们。应先把关联改回 ON a.customer_id = c.id，得到三条正确归属，再为修正后的 SQL 重新测量。旧耗时描述的是错误工作，不能拿它作为正确查询的可靠基线。

这里最需要带走的是一个能用于排错的顺序：先说清结果一行的含义，并用已知行核对；再检查完成这件正确工作所需的时间。计划不会替我们判断业务含义。

索引读得快一些，就应该留下吗？

索引随表一起维护。新增预约时，数据库既要写入预约行，也要加入这位客户对应的索引条目；删除预约或修改被索引的客户列时，也要更新索引。因此读取收益伴随额外空间和写入工作。单列客户索引不因修改任意非索引列就必然改写其索引键，应把维护成本连接到实际变动的列与行。

本例用很多不同客户值，把目标筛到 30 行。条件的选择性 selectivity 可以在这里理解为“这个条件能把范围缩小到什么程度”。不要仅数一列有几个不同值，还要看当前输入匹配多少行：一个大客户可能占全表多数，其他客户各只有几条，同一索引对不同输入的收益会不同。

status 在本例只有 confirmed、completed 两种，而且其他客户约一半属于每种状态。如果需求改成返回全部 confirmed 预约，即使按状态找到条目，也还要读取大量预约列。这样的单列索引可能收益很小，数据库也可能选择扫描。反过来，如果某个状态只占极少数，不能因为“只有两个状态”就一概断言索引无用。

是否保留索引，要把测得的读取收益放回调用频率和写入负担中判断。每天很少调用且已够快的读取，通常不值得仅为看到 SEARCH 增添结构；高频查询反复消耗大量时间时，同一收益就可能有实际价值。需要量化成本时，在独立文件数据库副本中比较页数或文件大小，并对相同批次写入计时。本章的内存读取程序没有测到这些成本，不能用它替生产环境作出完整承诺。

如果计划用了索引，但样本差异小到与波动相当，就如实保留“当前收益不明确”的结论。继续核对代表数据、输入分布与计时范围，或在已经满足应用要求时停止优化。建更多索引并不会自动减少这种不确定性。

回到完整应用时，哪些结论可以带走？

可以带走“固定正确结果、比较计划、同条件复测、考虑维护成本”的方法。C06 的四列表不能直接变成 C03/C04 的索引处方：在那里预约经 appointments.ticket_id → tickets.id，再经 tickets.customer_id → customers.id 找到客户。需要拿完整 JOIN、真实过滤与排序、代表数据重新取得计划，再提出对应索引候选。完整关系已经没有本章这根直接的 appointments.customer_id 列。

如果单列索引后仍在大量排序、反复回表取列，而且时间确实超过要求，再按需查看 composite index（多列索引）、covering index（所需列可以从索引取得）或 partial index（只为部分行建索引）。这些名字只提供进一步查找的入口；没有当前计划与实测时，不必先展开全部写法。解决后仍要回到相同结果与成本比较。

如果证据变成两个连接争用写入、等待锁或破坏名额规则，应回到事务与并发问题，按受控复现再查锁与 WAL；若项目确实迁移到服务器数据库，再核对该引擎的计划、账号和连接方式。这些机制不负责解释当前四列表的基本查找问题。`VACUUM` 面向需要验证的空间回收问题，也不是慢查询的通用修复命令。

本章停在一个可解释的判断上：这条查询仍返回同一组预约，当前计划改变了哪些访问工作，同样条件下测得多少收益，以及这些收益是否值得额外维护。以后重新遇到慢查询，应能从这几个关系找回工具，再查 SQLite CREATE INDEX的具体语法；只认得索引名称还不足以完成判断。

知识关系总览 | 把答案、事实和运行证据放回同一张图

学到这里，SQL 已经不是一长串用途相近的命令。它们分别改变答案的形状、约束可保存的事实，或组织一组状态变化。先把位置分清，再看现场为什么选择其中一个。

表的结构、查询的结果、Python 的值，不是三份同样的东西

长期事实

schema：有哪些表、列、身份与关系

constraints：哪些值和引用不能存在

已提交的记录：现在实际保存了什么

已提交的记录：读取

SELECT / JOIN / GROUP BY
↓
当前问题的结果表
↓
Python 列表、字典
↓
应用响应

已提交的记录：写入

INSERT / UPDATE / DELETE
↓
transaction 提交或撤回一组改变

一次应用操作旁边的证据

request_id、结果类别、行数、耗时
→ 发生了什么

正确结果、query plan、重复实测
→ 查询的代价是否值得优化

表结构不是表内数据；建好了 `appointments` 不代表已经有预约。查询结果也不是新的持久表：它可以只含三条预约，也可以只有一行统计。Python 拿到普通字典后，修改字典不会自动写回数据库。

这种区别会直接影响排错方向。查询结果少一行，先检查筛选与连接，不能据此断言存储丢了数据；响应里字段错了，可能是 Python 整理结果时改错，未必需要改表；写入程序打印成功，也必须有提交与读回证据，才能判断长期事实改变了。

一张答案怎样形成：先问“一行代表什么”

当前想回答什么	选择依据	容易混淆的近邻
哪些预约符合条件	`WHERE` 筛输入行，`ORDER BY` 与 `LIMIT` 定顺序及数量	限量不会自动给出最早、最近；显示三行也不证明只有三条符合

当前想回答什么	选择依据	容易混淆的近邻
预约属于哪位客户	`JOIN ... ON ...` 沿对应关系把需要的列放在一起	外键限制写入的引用，不替每条查询自动写连接条件
有哪些合格技术员	子查询提供编号，`IN` 判断成员	不需要技能明细时，连接所有技能反而可能放大后面的预约数
每位技术员有多少预约	`GROUP BY` 改成每组一行，再用聚合计算；`HAVING` 筛组	`DISTINCT` 去重不等于统计；`COUNT(*)` 会数左连接的占位行
每条预约在本技术员的安排里排第几	window 在分区内计算，保留每条预约明细	`GROUP BY` 会折叠明细，不能用已经折叠的两行代替三条预约
两份合格名单的共同成员	`INTERSECT` 组合同形状结果	`JOIN` 是关联事实，不是所有“把两份东西放一起”的通用动作

工具会组合使用，不是每个问题只能选择一个关键字。排班查询可以先筛技能，用 `LEFT JOIN` 保留零预约者，再分组计数。决定顺序的是答案需要的含义，不是命令名字听起来哪一个更高级。

未知值会穿过这些步骤。来源可能是原表尚未确定的时长，也可能是左连接没有匹配时补出的空位置。看到 `NULL`，先恢复它在当前位置的含义，再决定是否筛入、计数或向应用说明未知；不要一律改成零。

保存事实时，身份、约束与事务怎样接力？

客户 C1 是一个身份；邮箱是这个客户的当前联系信息。把邮箱只保存在客户表里，工单通过 C1 找到它，就减少了同一事实多处修改的矛盾。这个设计回答“事实由哪里保存”。

外键要求被引用客户存在，唯一约束防止重复占用，检查约束拒绝负时长，非空约束拒绝必须有值却缺失的情况。它们回答“哪些状态不能保存”。默认值只补上确定的初始状态，不会让未知事实凭空变真。

事务再向前一步：确认预约，不只是单独插入一条合法预约，还需要占用时段、留下事件。**约束保护每项不变量，事务保护这组相关改变的完整边界。** 因此第三步的错误语句被拒绝之后，应用仍然必须决定撤回前两步，不能捕获异常后照常提交。

事务也不包办所有问题。它不会替应用判断登录者有没有确认权限，不会把一个外部供应商操作自动并入本地提交，也不能仅凭一次单连接实验证明所有并发安排都安全。这些边界不清楚时，“用了数据库”仍然可能作出不可靠承诺。

查询、CTE 和 view：名字应当活多久？

一份只使用一次的查询，可以直接表达条件。查询内部需要给中间结果一个名字，例如“今天未取消的预约”，`WITH` 定义的 CTE 帮助分段阅读；它随这一条查询结束，不成为长期数据库接口。

看板和通知程序长期需要相同列名及取消规则时，view 才有稳定复用的理由。普通 view 保存查询定义，不是把数据复制成不会变化的快照；底表变动后，读取仍依赖定义能否正确解释当前结构。底表变更时同时调整 view，并在提交前验证公开列与结果，才能保护消费者。

至于谁能查看这些行，不由这个名字决定。SQLAlchemy 提供执行入口，参数绑定让输入留在数据位置，应用授权限定当前身份能看谁，数据库或文件权限限制进程可访问什么。它们守的是不同边界。一个参数化查询可以非常安全地执行一项未经授权的读取，所以“没有 SQL 注入”并不等于“没有越权”。

业务编号、request ID 与日志，分别回答哪一个“哪次”？

`T-42` 指明同一个工单，预约编号指明一条预约。客服先后两次读取同一工单，业务编号不变，却是两次处理；request ID 把其中某一次的入口、数据库操作与回应关联起来。

C04 的持久事件作为业务动作的一部分，与预约、时段同事务提交。C05 的运行日志则解释一次调用成功、失败以及用时。日志写了成功不替代数据库提交，数据库保存了事件也不自动提供所有运行耗时。

成功但结果为空，与查询失败还不知道有几行，也要分开。前者可以记零行，后者应该记未知并保留错误类别。日志需要的是足够定位的证据，不是客户姓名、完整 SQL 和全部输入；扩大记录内容还会带来泄露风险。

这些标识并不天然要求同一种字符串格式。C04 的 R-42 是局部事务实验的关联值，C05 的应用入口采用 UUID，并由 C06 的假设读取事件沿用。这是程序各自的输入约定；复用函数时必须服从函数的真实要求，不能仅凭字段都叫 `request_id` 就直接混用。

感觉慢，为什么不是立即加索引？

某个读取交回了 20 行，可能查错了客户，也可能漏掉日期并列时的排序。先核对“应当是哪些完整行及什么顺序”，才有稳定比较对象。

如果答案正确，再用同类操作日志确认值得调查的等待，用可重复数据记录基线；计划说明访问路线，实测说明这次环境中的代价。索引减少了一部分查找，也可能仍需排序。`SEARCH` 不是保证更快的成绩，`SCAN` 也不是必须消灭的错误。

C06 的单列索引只服务那个独立四列表中的客户条件。换回 C03、C04 的 `appointments → tickets → customers` 结构，需要对实际 JOIN 重新解释、测量；不能因为问题也叫“按客户查预约”，就把同一条索引命令原封不动移过去。

如果真正出现单列索引解决不了的稳定多列读取瓶颈，再进入高级索引的按需补充；如果出现可复现的两连接竞争，回到具体不变量补并发保护；如果项目决定换服务器数据库，再验证对应连接、账号、方言与事务。每条扩展都有返回当前读取或写入的任务，不必先学完全部数据库专题才能使用本册知识。

用一份完整实验把这些关系接起来

章内的小表让每一步能独立看清，配套的“社区维修数据实验室”则把查询、关系、约束、事务、应用读取和计划验证放到同一个可运行程序里。

先打开资料包根目录的可见入口 `01_配套代码_A-E`，进入 `examples` → `c_data_logs` → `repair-data`，把整个 `repair-data` 文件夹复制到自己选择的独立实验目录。这个入口与底层目录 `01_配套练习` 指向同一份配套文件。所有建立环境、改文件和运行都在这个副本中进行，不在资料包原目录里留下数据库、缓存或安装目录。

这份配套的结构不是 C04 数据库的直接迁移：预约直接保存客户、技术员和时段编号；标识使用 CUST-/TECH-/SLOT-/APT- 前缀，预约状态包括 open、confirmed、completed、cancelled。C03、C04 中的预约状态仍只有 confirmed、cancelled，查询路径仍经工单和时段。打开配套的 `schema.sql` 与 `seed.sql`，以其中的真实表和已知行为依据，不把两套 SQL 混着运行。

文件	阅读它时解决什么问题
schema.sql、seed.sql	数据结构是什么，实验从哪些已知记录开始
database.py	在哪里新建实验文件，每个连接怎样开启规则
queries.py	查询交回哪些行，输入怎样绑定，计划怎样记录
operations.py	多步写入、批量导入与取消怎样保护完整状态
test_*.py、milestone_check.py	哪条证据对应成功、失败或恢复，整条路线怎样一起运行

运行之前，先知道会改变什么

在实验副本中打开 PowerShell，确认当前目录能看到 `requirements.txt` 和 `milestone_check.py`。本册独立程序统一使用 Python 3.12 或更新版本，可先用 `python --version` 核对入口。配套自身允许的版本范围以其 README 为准；不应把它的较低最低版本用于含 `autocommit` 参数的 C03–C05 新程序。

然后建立局部环境、安装配套锁定依赖，再执行：

```powershell
python -m venv .venv
if ($LASTEXITCODE -ne 0) { throw '建立环境失败' }
& .\.venv\Scripts\python.exe -m pip install -r .\requirements.txt
if ($LASTEXITCODE -ne 0) { throw '安装依赖失败' }
& .\.venv\Scripts\python.exe -m pip check
if ($LASTEXITCODE -ne 0) { throw '依赖不一致' }
& .\.venv\Scripts\python.exe -m pytest -q
if ($LASTEXITCODE -ne 0) { throw '测试失败，先阅读对应失败详情' }
& .\.venv\Scripts\python.exe .\milestone_check.py
if ($LASTEXITCODE -ne 0) { throw '里程碑运行失败' }
```

`.venv` 与测试缓存保存在实验副本；配套的测试数据库在系统临时目录排他新建，不接受已有业务数据库作为练习目标。pytest 测试结束时会关闭连接，但测试目录可能按 pytest 的保留策略暂存；里程碑程序使用自己的临时目录，结束时自动清理。不要为了“省建库”把它改接到真实数据。

测试命令成功退出后，里程碑应输出含 `status: PASS` 的 JSON。再看里面分别记录了什么，而不只看末尾单词：已提交的预约标识，故意写坏事件后的完整回滚，脏导入行的位置，绑定查询的结果，以及建索引前后相同的答案。成功后预约、时段和事件的三处读回，由 `test_writing.py` 中的 `test_multistep_booking_commits_appointment_slot_and_event` 验证，不是每项完整状态都会展开在里程碑 JSON 里。

配套日志与 C05 的独立程序也不是同一个接口：前者使用 `duration_ms/error_category`，成功写为 `outcome=ok`，request ID 接收符合字符与长度要求的字符串；C05 使用 `elapsed_ms/error_type`，成功写为 `success`，函数接收 UUID 对象。两者对行数的含义一致：成功空结果是 `row_count=0`，成功非空结果记录实际行数，未能取得答案是 `row_count=null`。先看 `outcome`，再解释行数；不能把查询失败说成“查明没有记录”。其他字段相近，也不代表可以不读接口就合并统计。

配套的性能数字记录多次读取的合计，C06 独立程序记录单次样本的中位数；两者不是相同计时口径，不能把数字直接相除当成效果对比。各自应在相同程序、相同条件下比较前后。

怎样知道不是只把测试运行了一遍？

选择一个已经有明确证据的现象，例如“事件写入失败后，为什么时段仍然可用”。在代码里找到开始事务的位置、触发错误的位置和回滚后读回的位置；解释这些位置分别保证了什么，再改变一项合成输入，预测哪些状态应改变或不变。

若预测与结果不符，沿最早分叉处修正认识：是查询筛错行、连接改变行数、约束没开启、误把未提交当已提交，还是拿错了数据库文件？有现成的失败路线可用，不需要额外造一整套题。能沿证据解释并修改，比记住每个 SQL 拼写更接近实际使用能力。

官方练习可以作为更换数据后的迁移机会：需要练单表筛选就从 Querying 的题目里选择一项；需要验证表间关系就转到 Relating；写入和视图则在相应主题后选用。先看题目提供的 schema、目标及预期输出，不假设题库与维修实验相同。本书的完整代码属于原创实验，不是官方作业提交答案。相关题目入口在 CS50 SQL 各周材料。

下一册接走的是什么？

C 册现在交出了一层能保存事实、按条件回答问题、保护多步修改并留下运行线索的数据库能力。

如果确认预约还要通知外部供应商，会出现新的不确定性：请求超时，究竟是对方没有收到，还是已经处理但回应丢了？本地 `ROLLBACK` 不能把对方的操作自动撤回。D 册从这个边界继续，解释如何分类失败、限定等待与重试，并依靠持久记录判断同一业务动作接下来能安全做什么。

这里值得带走的不是某张表的全部列名，而是几种可以复用的判断：事实由谁保存，结果一行代表什么，成功要同时留下哪些状态，失败后凭哪项证据继续。它们就是把数据库放进完整应用时实际使用的结构认识。

来源与查用入口

本书的维修记录、程序与解释为自包含示例。课程主线参考 Harvard CS50 SQL 的 Querying、Relating、Designing、Writing、Viewing；事务属于官方 Optimizing 中的相关材料，在本书随多步写入使用。应用连接与运行日志按 Python、SQLAlchemy 的接口单独展开。

需要核对完整语法时，可查以下官方入口；阅读正文不依赖逐个打开它们：

- CS50 SQL 课程材料：讲次、练习要求与课程背景。
- SQLite 的 SELECT、表达式与聚合函数：查询结果、条件、空值与统计的语义。
- SQLite 的数据类型、外键与事务：普通表的存储行为、连接设置与提交边界。
- SQLite 的 view、WITH、window与查询计划：命名查询、保留明细的计算及访问路线。
- Python sqlite3与 SQLAlchemy 2.0 连接和事务：程序入口、绑定值及事务控制。

这些链接是持续更新的官方页面，不是固定版本快照。实际运行依赖文件给出的版本和本章明确写出的 Python 条件；升级、迁移或更换数据库时，应在独立实验中重新核对成功、失败与恢复，不能只因接口名字相同就认为行为未变。

本册总览（需要时再回看）

这里保留整册地图、章节关系和查用入口；第一次阅读不需要先读完。

本册知识结构总览

这不是六组互不相干的 SQL 命令，而是围绕同一批长期事实的几种责任：

应用收到一个业务问题或修改请求	
需要回答问题	从单表选行、选列、排序、汇总 C01 沿标识组合不同表的事实 C02 由绑定查询或稳定 view 交回应用 C05
需要改变事实	表结构和约束界定什么数据可以存在 C03 写入放进 transaction，同成或同败 C04
需要判断运行情况	日志关联操作、结果和耗时 C05 结果先正确，再用 plan 和实测判断索引 C06

其中有几条会反复相遇的关系。

需要分清什么	它们怎样配合
存储的事实与查询的答案	表保存记录；同一批记录能回答不同问题，结果的一行不一定仍代表一条原记录
表间关系与查询连接	主键、外键描述记录怎样对应；`JOIN` 根据条件组合当前查询需要的行
约束与事务	约束拒绝不合法状态；事务把一组修改作为整体提交或撤回，彼此不能替代
绑定值与查看权限	绑定让输入保持为数据；应用仍要决定当前调用者能否查看这个客户的资料
业务数据与日志	数据库保存当前事实；日志解释一次操作的经过，不能靠写了“成功”来补出未提交的数据

需要分清什么	它们怎样配合
正确性与性能	先确定结果应是什么，再判断取得这个结果的代价；更快的错误答案仍然错误

这些区别现在不必全记住。先认出本册围绕“答案、长期事实、修改边界、运行证据”展开；遇到具体问题，再回到对应位置。需要整理工具选择时，再回到本册后部的“知识关系总览”，用已经看过的结果重新梳理。

从 Python 和 HTTP 接过来的是什么？

Python 函数可以取得参数、处理并返回值。HTTP 让两个程序能交换请求与响应。本册往这条路线中补上持久数据：

> HTTP 请求里的输入
> → 应用判断输入与权限
> → 带着明确条件读取或修改数据库
> → Python 得到结果，形成响应

这里的数据库是受应用调用的一层，不替应用决定全部业务规则，也不直接等同于 HTTP 服务。

例如 B 册示例把工单存在运行中程序的内存字典里，重启便会重新初始化。这里使用 SQLite：它是数据库引擎，能在本机文件中管理表和已提交的数据，不需要另外启动数据库服务器；SQL 是向引擎表达读取、结构与修改要求的语言。后面的 Python `sqlite3` 与 SQLAlchemy，是程序调用数据库的入口，不是另外两种数据格式。

早期纸面实验会使用内存 SQLite 数据库，关闭就消失，便于安全重做；需要验证持久化时才使用独立临时文件。**能读写内存示例，不等于已经证明数据在重启后仍存在。** 本册会把这两种证据分开。

怎样读这些示例？

每章先读问题与已知记录，沿结果看“为什么恰好留下这些行”。整段建表或运行程序放在需要实际验证的位置，它们不是必须先默写的入门门槛。

常用的筛选、连接和参数化读取，值得在后续运行中逐渐用顺；能解释列名却不能判断查询会返回谁，还没有理解这条查询。完整迁移脚本、连接配置或查询计划的格式可以查用，关键是清楚它保护什么、执行后该查看什么证据。

C01、C02 使用小型读取模型；C03、C04 共用一套完整业务结构；C05、C06各自采用说明当前问题所需的独立实验。相同表名不保证列也相同。每次切换都会明说，不应把不同章节 SQL 直接拼进同一个数据库。最后的配套程序提供另一套完整、可重复运行的练习库，并说明如何对应这些关系。